Homomorphic Encryption for High-precision Arithmetic of Approximate Numbers

Shihe Ma, Jiaming Liu, Anyu Wang, and Xiaoyun Wang

Tsinghua University, Beijing, China
msh24@mails.tsinghua.edu.cn, liujm25@mails.tsinghua.edu.cn
anyuwang@tsinghua.edu.cn, xiaoyunwang@tsinghua.edu.cn

Abstract. The CKKS fully homomorphic encryption scheme enables efficient arithmetic over complex numbers, but each homomorphic multiplication consumes a large modulus to ensure enough precision, limiting both the circuit depth and precision supported by CKKS. This work addresses this problem by proposing Generalized CKKS (GCKKS), a CKKS analogue of the GBFV scheme (EUROCRYPT 2025) that trades the number of slots for reduced modulus consumption.

Despite its high-level similarity to GBFV, GCKKS faces new challenges caused by the different encoding and encryption structures of CKKS. We address these challenges with novel techniques not present in GBFV. Our plaintext encoding starts with an improved and SIMD version of Costache, Smart, and Vivek's encoding in 2017. Namely, it rounds complex values to cyclotomic integers instead of Gaussian integers, and reduces the rounding problem to solving CVP instances. This encoding allows storing high-precision complex values in polynomials with small canonical norms. As an important observation, the encoding can be simplified to a sparse CKKS encoding followed by multiplication with a polynomial constructed from a short vector in the encoding lattice. We show that by integrating this encoding format into GCKKS decryption, most GCKKS homomorphic operations and the corresponding noise analyses reduce to their CKKS counterparts. This relieves GCKKS from any heavy computational or noise overhead.

In experiments, GCKKS provides 7 levels with 1000-bit precision at ring dimension 2^{14}, where each multiplication takes less than 30 ms. To our knowledge, this precision has never been achieved before. It can be used to preserve enough precision after noise flooding, a necessary ingredient for CKKS circuit privacy and a common approach to secure threshold CKKS and to provide IND-CPA^{D} security.

Keywords: CKKS· High precision · Lattice · GCKKS · Homomorphic Encryption.

1 Introduction

Homomorphic encryption (HE) enables computation over encrypted data and is a fundamental tool for privacy-preserving computation. Existing HE schemes

mainly fall into three categories: BGV [12] and BFV [10,34] for exact arithmetic over modular integers or finite fields, CKKS [23] for approximate arithmetic over complex numbers, and FHEW [30] and TFHE [24,25] for Boolean or small-integer arithmetic. Among them, CKKS is particularly well suited to batched, deep homomorphic computation over real- or complex-valued data and thus plays a distinctive role in the HE toolbox.

However, CKKS must balance precision against multiplicative depth. To maintain precision against noise from encoding, encryption, and homomorphic operations, CKKS scales plaintext values by a large factor Δ during encoding. Each homomorphic multiplication squares the plaintext magnitude (called the *scale*). Without intervention, the scale grows as $\Delta^2, \Delta^4, \Delta^8, \ldots$ and quickly exhausts the ciphertext modulus Q, even for a shallow circuit.

The *rescaling* operation controls this growth by restoring the scale to Δ after each multiplication. It divides the scale and ciphertext modulus by the same factor, so they evolve as

$$(\Delta, Q) \xrightarrow{\texttt{Mult}} (\Delta^2, Q) \xrightarrow{\texttt{Rescale}} (\Delta, \frac{Q}{\Delta}) \xrightarrow{\texttt{Mult}} (\Delta^2, \frac{Q}{\Delta}) \xrightarrow{\texttt{Rescale}} (\Delta, \frac{Q}{\Delta^2}) \longrightarrow \cdots .$$

Rescaling therefore enables deep CKKS circuits, but each multiplication still consumes about $\log_2(\Delta)$ bits of ciphertext modulus. Typical choices of Δ range from 2^{40} to 2^{60} [9], and can reach 2^{80} to 2^{100} in high-precision settings [22,1]. Even larger Δ is required to ensure enough precision after noise flooding, which is both necessary for strong CKKS circuit privacy [39,47] and commonly[1] used to secure threshold and multikey CKKS [2,17,16,47] and to provide IND-CPA^{D} security for CKKS [49]. This rapid consumption limits the multiplicative levels between bootstrapping operations, constrains very-high-precision computation, and forces larger RLWE dimensions to accommodate the required modulus at a fixed security level.

Prior approaches reduce this cost mainly by modifying the plaintext representation or ciphertext structure. For plaintext representation, Costache et al. obtained a small-norm encoding of a complex number by rounding it to a cyclotomic integer in $\mathbb{Z}[\zeta_{2n}] \cong \mathbb{Z}[X]/(X^n + 1)$, then encrypting the result with BGV under a sufficiently large plaintext modulus [27]. Building on this representation, Bootland et al. used the plaintext space $\mathbb{Z}_{b^{n/m}+1}[\zeta_{2m}] \cong \mathbb{Z}[X]/(X^n + 1, X^m + b)$ in BFV to support a larger plaintext modulus [8]. Chen et al. extended the technique to SIMD by using $\mathbb{Z}_{b^{n/m}+1}[\zeta_{2m}]$ to simulate arithmetic over $\mathbb{R}[\zeta_{2m}] \cong \mathbb{C}^{m/2}$, at the cost of losing the small-norm advantage [18]. Bonte et al. and Castryck et al. instead represented complex numbers and other data types as Laurent polynomials in BFV slots, following a similar idea [7,13].

For ciphertext structure, Cheon et al. proposed Tuple-CKKS, which decomposes a standard CKKS ciphertext into one high-part ciphertext and $t - 1$ low-part ciphertexts using homomorphic Euclidean division [21]. Roughly, this reduces the modulus bits consumed per multiplication by a factor of t, while requiring t relinearizations, t rescalings, and $t(t + 1)/2$ tensor products.

[1] Large noise flooding can be avoided in certain methods [43,52].

However, these approaches have significant limitations for deep, high-precision homomorphic computation. Cyclotomic-integer and Laurent-polynomial methods rely on BGV/BFV rather than CKKS to prevent RLWE noise from corrupting encoded messages. Therefore, rescaling requires expensive BGV/BFV operations, such as downscaling the plaintext polynomial or dropping Laurent-polynomial terms. The absence of efficient rescaling limits these methods to shallow circuits. Additionally, the cyclotomic-integer methods either do not support SIMD packing or lack operations for inter-slot data movement.

For Tuple-CKKS, its latency grows at least linearly with t, effectively limiting t to 2 in practice. Moreover, fresh Tuple-CKKS ciphertexts still have scale Δ, consuming a significant portion of the ciphertext modulus at high precision. Additionally, one noise term grows rapidly with multiplicative depth and reduces precision in deep circuits. Together, these costs substantially restrict the practical gains of Tuple-CKKS in deep, high-precision computation.

Thus, existing methods for reducing modulus consumption in homomorphic computation of complex-valued data either lack an efficient rescaling method or introduce substantial overhead in homomorphic multiplication. In contrast, the GBFV scheme of Geelen and Vercauteren [37] and MatriGear by Cha et al. [15], which generalize both BFV and CLPX by Chen et al. [19], reduces modulus consumption for high-precision BFV computation without additional computational overhead. It achieves this reduction through a generalized encoding that trades slots for slower noise growth: halving the slot count roughly halves the modulus consumption in bits. GBFV remains compatible with existing BFV operations and enables large cyclotomic primes as plaintext moduli while keeping the RLWE dimension small. This result naturally raises the following question:

Can CKKS achieve a similar slot-count/modulus-consumption tradeoff with little computational overhead while preserving its standard homomorphic arithmetic?

1.1 Our Contribution

We answer this question affirmatively with Generalized CKKS (GCKKS), a generalization of CKKS inspired by GBFV that also improves the encoding of Costache et al. GCKKS trades slots for lower modulus consumption: roughly, each halving of the slot count halves the number of modulus bits consumed per multiplication. At the same time, it supports all standard CKKS homomorphic arithmetic with little or no computational overhead. We complement the scheme with rigorous analyses and experimental validation of both the encoding-noise bound and the noise growth during homomorphic computation.

This reduction in modulus consumption translates into larger multiplicative depth for a fixed modulus budget and target precision, support for ultra-high-precision arithmetic, and smaller RLWE dimensions while retaining sufficient depth. It also provides modulus savings at no additional cost when plaintexts are sparsely encoded. As a concrete demonstration, our parameter set at ring

dimension $N = 2^{14}$ provides seven multiplication levels. Its initial square-and-rescale preserves more than 1000 bits of precision and takes 28.16 ms on average. Such ultra-high precision is particularly useful to preserve enough message precision against the exponentially large noise introduced by noise flooding.

1.2 Technical Overview

The two core ingredients for GCKKS are a generalized plaintext encoding and a new ciphertext decryption format. The former generalizes CKKS encoding and improves and simplifies Costache et al.'s lattice-based cyclotomic-integer encoding. For $w = 2^{-p}$ and $k > 4$, it encodes $2N/k$ complex values with roughly p-bit precision and a canonical norm of $O(2^{4p/k})$. Importantly, we show how to implement this encoding as a sparse CKKS encoding followed by multiplication by a polynomial constructed from a short vector in the encoding lattice. This simplified encoding format is the key to efficient homomorphic operations.

We augment GCKKS decryption by multiplying the standard decryption result by the same polynomial used in encoding. Under this modified decryption, a CKKS ciphertext with a p-bit scaling factor becomes a GCKKS ciphertext with a scaling factor of roughly $4p/k$ bits. This creates a link between CKKS and GCKKS: noise introduced by encryption and homomorphic operations in the CKKS domain is automatically *rebalanced* when entering the GCKKS domain. The rebalanced noise terms have magnitudes comparable to those of the encoding errors and therefore preserve the message precision. Finally, we show that the unused slots introduce new challenges for noise control and develop a noise model that stabilizes their magnitudes during homomorphic operations.

Plaintext encoding. Let $J = \sqrt{-1}$ denote the imaginary unit. Standard CKKS encodes $N/2$ complex values into a polynomial in $\mathbb{Z}[J][X]/(X^{N/2} - J) \cong \mathbb{Z}[X]/(X^N + 1)$ through the inverse canonical embedding [40]:

$$\mathbb{C}^{N/2} \xrightarrow[\sim]{\texttt{iDFT}} \mathbb{C}[X]/(X^{N/2} - J) \xrightarrow{\substack{\text{entrywise} \\ \text{round}}} \mathbb{Z}[J][X]/(X^{N/2} - J) \cong \mathbb{Z}[X]/(X^N + 1).$$

We generalize this encoding by replacing $J = \zeta_4$ with a generic ζ_k with $k > 4$:

$$\begin{array}{ccccccc} \mathbb{C}^{2N/k} & \xrightarrow[\sim]{\texttt{iDFT}} & \mathbb{C}[X]/(X^{2N/k} - \zeta_k) & \xrightarrow{\substack{\text{entrywise} \\ \text{round}}} & \mathbb{Z}[\zeta_k][X]/(X^{2N/k} - \zeta_k) & \cong & \mathbb{Z}[X]/(X^N + 1) \\ \boldsymbol{\Delta}\mathbf{v} & \mapsto & m' & \mapsto & m & \mapsto & M. \end{array}$$

The first tradeoff in the encoding is the *slot/precision tradeoff*: increasing k trades the number of valid slots for higher encoding precision. To see why, consider the subset of $\mathbb{Z}[\zeta_k]$ with coefficients bounded by some $B > 0$:

$$\mathbb{Z}[\zeta_k]_{\le B} = \Big\{ \sum_{i=0}^{k/2-1} a_i \zeta_k^i : |a_i| \le B \Big\}.$$

Increasing either B or k while fixing the other will increase the density of $\mathbb{Z}[\zeta_k]_{\leq B}$, leading to a smaller rounding error.

There is also a *precision-norm* tradeoff in the slots of M: decreasing the encoding error in useful slots will increase the magnitude of the unused slots. The reason is that a smaller useful-slot encoding error requires a smaller rounding error from $\mathbb{C}$ to $\mathbb{Z}[\zeta_k]$. Then, according to the previous example of $\mathbb{Z}[\zeta_k]_{\leq B}$, $\mathbb{Z}[\zeta_k]$ elements produced by rounding will have larger coefficients. This directly translates to larger coefficients in M via the last isomorphism in our encoding, leading to larger unused-slot values. Since we want to minimize the canonical embedding norm of M to minimize the noise growth during homomorphic multiplication, this increased magnitude of unused-slot values is undesirable.

With this precision-norm tradeoff in mind, we reduce the rounding of $m'[i]$ to solving CVP instances in $\mathcal{L}(\mathbf{B}_{k,w})$ using the following basis and target:

$$\mathbf{B}_{k,w} = \begin{pmatrix} \mathrm{Re}(\zeta_k^0) & \cdots & \mathrm{Re}(\zeta_k^{k/2-1}) \\ \mathrm{Im}(\zeta_k^0) & \cdots & \mathrm{Im}(\zeta_k^{k/2-1}) \\ w & & \\ & \ddots & \\ & & w \end{pmatrix}, \qquad \text{CVP target} = \begin{pmatrix} \mathrm{Re}(m'[i]) \\ \mathrm{Im}(m'[i]) \\ \star \\ \vdots \\ \star \end{pmatrix}$$

where w is a very small positive parameter for adjusting the precision-norm tradeoff, with smaller w preferring higher precision and larger canonical norms.

Although it is natural to fill all $\star$'s in the CVP target with zeros, as Costache et al. did[2], this choice is not optimal. This CVP target has a non-negligible distance to $\mathrm{span}(\mathbf{B}_{k,w})$, causing additional encoding error. Instead, we find that the *optimal* CVP targets are naturally given by the unique $M' \in \mathbb{R}[X]/(X^N+1)$ whose useful slots are $\Delta\mathbf{v}$ and whose unused slots are all zero. This polynomial can be obtained from m' through a CRT map $\mathtt{Lift}_{N,k} : \mathbb{C}[X]/(X^{2N/k} - \zeta_k) \to \mathbb{R}[X]/(X^N+1)$, defined as

$$M' \equiv m' \bmod X^{2N/k} - \zeta_k, \qquad M' \equiv 0 \bmod \frac{X^{N/2} - J}{X^{2N/k} - \zeta_k}. \tag{1}$$

The CVP target for $m'[i]$ is then constructed as

$$\begin{pmatrix} \mathrm{Re}(m'[i]) & \mathrm{Im}(m'[i]) & wM'[i] & wM'[i+2N/k] & \cdots \end{pmatrix}^T \in \mathrm{span}(\mathbf{B}_{k,w}).$$

The encoding is realized by solving $k/2$ CVP instances in $\mathcal{L}(\mathbf{B}_{k,w})$, one for each $m'[i]$. The quality of encoding is given by the following result.

Result 1. *(Informal) The useful-slot encoding errors and unused-slot values in CVP-based encoding follow complex Gaussian distribution with standard deviations of $O(w^{1-4/k})$ and $O(w^{-4/k})$, respectively.*

Taking $\Delta = O(w^{-4/k})$ directly gives an encoded polynomial M with $-\log_2 w$ bits of precision and a canonical norm of $O(w^{-4/k})$.

[2] They turned the CVP into an SVP through Kannan embedding [44].

A simpler encoding algorithm. We present a simpler and more efficient encoding process that does not require any online lattice computation. The core component is a polynomial F constructed from a short vector in $\mathcal{L}(\mathbf{B}_{k,w})$ with special canonical embedding values.

Result 2. *(Informal) There exists a polynomial $F \in \mathbb{Z}[X^{2N/k}]/(X^N+1)$, which has the same value c_m in every useful slot, with $|c_m| = O(w^{1-4/k})$. Its unused slots have magnitudes of $O(w^{-4/k})$.*

Denote by $\texttt{Lift}$ the CRT map sending m' to M' in (1). The simplified encoding process outputs

$$\lfloor \texttt{Lift}(\texttt{iDFT}(\Delta c_m^{-1}\mathbf{v}))\rceil \times F. \tag{2}$$

That is, we first obtain a CKKS plaintext encoding $\Delta/c_m \cdot \mathbf{v}$ in useful slots and zeros in unused slots, then multiply it by F. By Result 2, setting $\Delta = O(w^{-4/k})$ ensures that multiplication by F restores the useful-slot values $\Delta\mathbf{v}$ and *rebalances* all canonical-embedding values to the same magnitude of $O(w^{-4/k})$. This process is visualized in Figure 1.

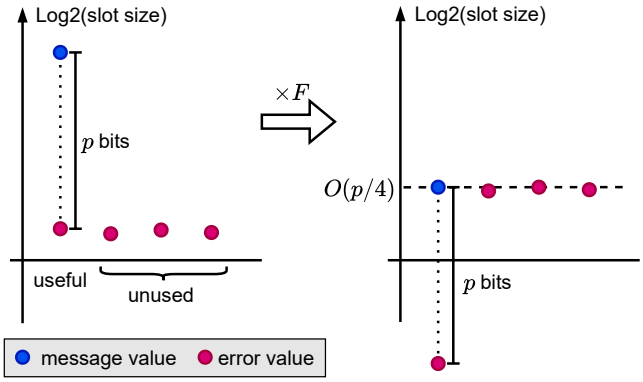


Fig. 1. Effect of multiplying by F on the message and encoding noise size, with four slots and $k = 16$.

Homomorphic operations. The ability to encode with high precision and a small canonical norm does not naturally imply high-precision homomorphic operations. The reason is that encryption and modulus switching introduce noise of magnitude $\Omega(\sqrt{N})$ into every slot, overwhelming the much smaller encoding errors in the useful slots.

We solve this problem by incorporating the mult-by-F encoding format into ciphertext decryption. Namely, a complex vector $\mathbf{v}$ is encrypted as

$$(b = -as + M + e_{\text{enc}}, a) \in \left(\mathbb{Z}_q[X]/(X^N+1)\right)^2, \text{ with } M = \lfloor \texttt{Lift}(\texttt{iDFT}(\Delta c_m^{-1}\mathbf{v}))\rceil,$$

which has a F-augmented decryption format of

$$\mathtt{Dec}_{s,F}((b,a)) = (b+as)F \equiv MF + e_{\text{enc}}F \bmod q,$$

where MF is exactly the encoded polynomial in (2), and by Result 2, $e_{\text{enc}}F$ has small enough magnitudes in the useful slots to preserve the precision. Note that even if M wraps around q, $\mathtt{Dec}_{s,F}$ remains correct as long as $\|MF\|_\infty$ is small compared to q. This ineffective q-overflow of M is called pseudo overflow [5].

Intuitively, this F-decryption creates a link between CKKS and GCKKS ciphertexts. Consider the GCKKS ciphertext above with useful-slot values $\Delta\mathbf{v}$. Under CKKS decryption, it has useful-slot values of $\Delta\mathbf{v}/c_m$ and near-zero unused-slot values (up to pseudo overflows). Any homomorphic noise introduced into the CKKS slots is automatically rebalanced in the GCKKS slots by multiplication by F in GCKKS decryption. Therefore, *the useful-slot relative noise growth of a GCKKS ciphertext with scale $\Delta = O(w^{-4/k})$ matches that of a CKKS ciphertext with scale $\Delta/|c_m| = O(w^{-1})$.*

Another consequence of this link is that most GCKKS homomorphic operations have implementations identical to their CKKS counterparts and thus have identical computational costs. The only two exceptions are homomorphic multiplication and conjugation, because they break the F-decryption format and require extra operations to restore it. We will later show how to implement them with limited overhead.

The last component of the GCKKS scheme is a noise-control model for the growth of unused-slot magnitudes during homomorphic computation. Based on this model, a sufficiently large ciphertext scale is calculated to stabilize the unused-slot magnitudes throughout homomorphic computation, so that they do not diverge to infinity. Result 3 gives the minimum sufficient ciphertext scale.

Result 3. *(Very informal) To obtain a relative useful-slot noise growth equal to standard CKKS with scale Δ', the minimum sufficient GCKKS scale is*

$$\Delta_{\min} = O\left(\max(\delta_\beta^{(k-2)/2}(\sqrt{kNLh})^{1-4/k}(\Delta')^{4/k}, \delta_\beta^{(k-2)/2}\sqrt{kNLh})\right),$$

where δ_β is the root Hermite factor for β-BKZ, L is the maximum supported linear transformation size, and h is the secret Hamming weight.

We do not propose GCKKS bootstrapping in this paper but leave it as an important future work.

1.3 Other Related Work

Discrete CKKS, originally designed for Boolean or low-precision integer arithmetic over CKKS [29,3,4], has recently been used to perform high-precision integer arithmetic in CKKS. These approaches either decompose a large integer into smaller numbers before storing them in slots [6,45,46,14], or simulate arithmetic in special rings [11] by storing the canonical-embedding values of ring elements in slots [36]. Although both discrete CKKS and our scheme pursue high-precision

arithmetic using CKKS, they handle fundamentally different data types. Discrete CKKS focuses on *exact arithmetic on modular integers*, whereas GCKKS aims for *approximate arithmetic on complex-valued data*. Therefore, they are designed for different computation tasks and cannot be meaningfully compared.

1.4 Parallel and Follow-up Work

In a recent work parallel with this work, Dumezy independently developed similar techniques [31]. Their symbols can be related via $g = F$ and $R = k/4$, and we use the symbols F and k to be consistent with the other part of this work.

The differences between these two works mostly lie in three aspects.

Ciphertext format. Their ciphertext absorbs the decryption factor F into the stored RLWE components. An encryption for some encoded polynomial m is then

$$(b = -as + Fe + m, a).$$

Consequently, they use a different algorithm to rescale a polynomial $u \in \mathbb{Z}[X]/(X^N + 1)$. The rescaling of u by a factor of r is given by

$$h = [F^{-1}u]_r, \qquad \mathrm{Round}_{F,r}(u) = \frac{u - Fh}{r},$$

where the inverse of F is calculated modulo r.

These two ciphertext formats are closely related. When F is invertible modulo every involved modulus, the two ciphertext formats are interchangeable since

$$(-as + Fe + m, a) \equiv (-a's + e + F^{-1}m, a') \times F \bmod Q,$$

where $a' = F^{-1}a$. Their shaped rounding algorithm also becomes ordinary rounding via

$$\mathrm{Round}_{F,r}(Fu) = \lfloor \frac{u}{r} \rceil F.$$

Despite their close relationship, the two ciphertext formats lead to different advantages and disadvantages.

- **Requirement of F.** Dumezy's ciphertext format requires F to be invertible modulo every involved modulus in order to reduce the security of their scheme to RLWE and to make $\mathrm{Round}_{F,r}$ well-defined. In contrast, such restriction is absent in GCKKS, because GCKKS has standard RLWE ciphertexts and does not rely on the invertibility of F modulo some integer to define the homomorphic operations.
- **Evaluation key reuse.** GCKKS reuses both the standard CKKS evaluation keys and the key-switching algorithm without modification, while automatically shaping the resulting noise through F-decryption. Under their ciphertext format, the reused keys do not have shaped (or re-balanced) error, so their scheme needs to either use large enough auxiliary modulus P

to keep the contribution of these noises small, or to generate additional key-switching keys with shaped noises ge when the precision is too high. They also described an alternative that reuses ordinary evaluation keys by converting to the GCKKS format, performing key switching, and converting back.

– **Performance of homomorphic operations.** The affected homomorphic operations include rescaling, key-switching, ciphertext multiplication, and homomorphic conjugation.

  Ciphertext rescaling. GCKKS rescaling does not incur additional overhead. In contrast, their method requires two additional coefficient-domain polynomial multiplications with F or F^{-1} per RLWE component, i.e., a total of four. These multiplications usually fit into word size for practical parameters. Since F only has $k/2$ nonzero coefficients, each ciphertext rescaling requires $2kN$ word-size integer multiplications.

  Key-switching. GCKKS does not introduce additional overhead, while the potential overhead of their method comes from two aspects. The first is using larger P, converting to and from GCKKS ciphertexts, or requiring extra key-switching keys with shaped noise. For the settings in their implementation, ordinary evaluation keys already give a negligible noise contribution, so no additional treatment of evaluation-key noise is needed. The other is from the rescaling used in key-switching, whose performance issue has been discussed above.

  Ciphertext multiplication. The overhead of their method comes from the rescalings it uses. Meanwhile, the overhead of GCKKS comes from additionally multiplying the tensored result with F in the NTT domain, costing $3lN$ word-size multiplications with l the number of RNS primes.

  Homomorphic conjugation. Their method has no additional overhead if we exclude the related key-switching from discussion. However, GCKKS needs an additional level of plaintext-ciphertext multiplication for non-unit F, or a level-free plaintext multiplication in NTT domain when F is a unit.

  Remarkably, the two ciphertext formats place their additional computational costs in different operations, namely shaped rescaling in their scheme, and the correction by F during ciphertext multiplication in GCKKS. However, the additional overhead in GCKKS's homomorphic conjugation is only a result of the construction in this work, but not a necessary consequence of the ciphertext format. For example, it is possible to make the homomorphic conjugation in GCKKS overhead-free by restricting F to be conjugate-invariant, at the cost of a half-bit degradation in the expected encoding quality[3].

Search of F. For fixed k, both works search for F with useful-slot size $O(w^{-4/k+1})$ and unused-slot size $O(w^{-4/k})$, but following different approaches. Our work searches for F in the $\mathbf{B}_{k,w}$ lattice and produces F that is not necessarily a unit, whereas their work searches for a unit F in the log-unit lattice [28]. Using a unit F is a convenient way to make it invertible modulo any integer, and serves

[3] This will be discussed in our follow-up work.

as a sufficient condition to meet the requirements of their ciphertext format. According to our numerical experiments in Appendix E, the two types of F are identical when $k = 8$, and have similar encoding quality when $k \leq 64$. But for larger values of k like $k = 256$, with the enumeration-based search used in our experiments, finding a unit F with an encoding quality as good as those from $\mathbf{B}_{k,w}$ becomes much harder, while finding F from $\mathbf{B}_{k,w}$ remains relatively efficient.

Plaintext encoder. A final difference is that the two schemes use different plaintext encoders. Suppose F is a unit. Dumezy's work and ours encode the noiseless plaintext polynomial m_0 respectively as

$$\mathrm{Round}_{F,r}(\lfloor r\Delta m_0 \rceil), \qquad F\lfloor \Delta F^{-1} m_0 \rceil.$$

Using the equality $\mathrm{Round}_{F,r}(u) = F\lfloor \frac{F^{-1}u}{r} \rceil$, the encoding results become

$$F\lfloor \frac{F^{-1}\lfloor r\Delta m_0 \rceil}{r} \rceil, \qquad F\lfloor \Delta F^{-1} m_0 \rceil.$$

The two encoding results are only equal when the inner rounding error $F^{-1}\frac{\lfloor r\Delta m_0 \rceil - r\Delta m_0}{r}$ is small enough so that it does not affect the outer rounding.

Bootstrapping. A bootstrapping procedure based on an idea similar to that of GBFV [38] is described and implemented in Dumezy's work, while no bootstrapping is given for GCKKS. Yet, we have been working on GCKKS bootstrapping and have finished most of the design. The main idea is summarized below.

Preview of GCKKS bootstrapping. Both GBFV and GCKKS represent a small number of high-precision values by 'flattening' them into a higher dimension. However, both the GBFV and Dumezy bootstrapping procedures concentrate the useful data into a cyclotomic subring, thereby restoring the standard BFV/CKKS encoding somewhere in the bootstrapping. This is useful to reduce the number of data values that needs to be processed during bootstrapping, and is efficient when the precision is not too large. However, this design also makes their bootstrapping unable to operate in ultra-high precision (e.g., when the number of precision bits is close to the modulus bit-length), which is the regime our bootstrapping targets.

The core idea is that we should preserve the flattened representation of useful data, and only bootstrap the overflow I. We realize this idea upon the high-precision bootstrapping framework by Choe et al. [26]. Namely, we modup the ciphertext and bit-decompose the coefficients of I in low precision, iteratively double the encoding precision of these bits, and linearly combine them to cancel the effect of I in the useful slots. The precision doubling stage involves converting between different F to avoid the high modulus consumption in the original design by Choe et al., and each stage splits a ciphertext into two ciphertexts with doubled precision. However, this will invalidate the existing unused-slot noise

control, and we solve this by developing a slot-aware noise analysis and designing a better search for F that works directly over the canonical embedding.

We have finished the sections for improved encoding and ciphertext conversion, and have sketched the bootstrapping section. We plan to finish a draft version of the work (without an implementation) and upload it to IACR ePrint in a few days.

2 Preliminaries

2.1 Basic Notation

Let $\mathbb{Z}$, $\mathbb{R}$, and $\mathbb{C}$ denote the integers, real numbers, and complex numbers, respectively. We write $J = \sqrt{-1}$ for the imaginary unit and $\zeta_n = \exp(2\pi J/n)$ for the primitive n-th root of unity with argument $2\pi/n$. The complex Gaussian distribution is denoted by $\mathcal{CN}(\mu, \sigma^2)$. For a positive integer n, let $[n] = \{0, 1, \ldots, n-1\}$.

Throughout this work, N is a power of two, and $R = \mathbb{Z}[X]/(X^N + 1)$ is the $2N$-th cyclotomic integer ring. For an integer modulus $q \geq 2$, let $R_q = (\mathbb{Z}/q\mathbb{Z})[X]/(X^N+1)$. Every element of R_q is represented by its unique polynomial of degree less than N, and $f[i]$ denotes the coefficient of X^i in this representative.

Vectors and matrices are written in boldface, e.g., $\mathbf{v}$ and $\mathbf{B}$. Their entries are indexed from zero, with $\mathbf{v}[i]$ standing for the i-th entry of $\mathbf{v}$. We use $(\cdot)^T$ for transpose and $\|\cdot\|_p$ for the usual ℓ_p norm of a real or complex vector. For a polynomial, $\|f\|_p$ denotes the ℓ_p norm of its coefficient vector.

For $f \in \mathbb{R}[X]/(X^N + 1)$, its canonical embedding (with one element chosen from each conjugate pair) is

$$\tau(f) = \big(f(\zeta_{2N}^{5^i})\big)_{i \in [N/2]} \in \mathbb{C}^{N/2}.$$

For $S \subseteq [N/2]$, we define the restricted canonical embedding and canonical norm

$$\tau_S(f) = \big(\tau(f)[i]\big)_{i \in S}, \ \|f\|_{\mathrm{can},S} = \|\tau_S(f)\|_\infty,$$

and use $\|f\|_{\mathrm{can}} = \|f\|_{\mathrm{can},[N/2]}$ for the standard canonical norm. When S is the set of useful or unused slots, we replace S in the subscript by useful or unused, respectively.

2.2 RLWE Encryption

Let χ_s and χ_e be distributions in R with small polynomial coefficients. An RLWE secret key is a polynomial $s \leftarrow \chi_s$. To encrypt a plaintext polynomial $M \in R_q$ under s, sample a uniformly from R_q and $e \leftarrow \chi_e$, and output

$$c = (b, a) = (-as + M + e, a) \in R_q^2.$$

Decryption computes $\mathtt{Dec}_s(c) = b + as = M + e \pmod q$. We call e the decryption noise. The RLWE ciphertext above is denoted by $\mathrm{RLWE}_{s,q}(M; e)$. The Ring Learning with Errors (RLWE) assumption states, informally, that the ciphertext is computationally indistinguishable from a pair of uniformly random ring elements [50].

2.3 CKKS Scheme

CKKS is an RLWE-based homomorphic encryption scheme for approximate arithmetic over complex numbers [23]. It packs up to $N/2$ complex values into one plaintext and evaluates arithmetic slot-wise in a single-instruction-multiple-data (SIMD) manner.

Encoding and encryption. Let $\mathbf{v} \in \mathbb{C}^{N/2}$ be a slot vector and let $\Delta > 0$ be its scaling factor. CKKS first applies the inverse canonical embedding to obtain

$$M' = \tau^{-1}(\Delta \mathbf{v}) \in \mathbb{R}[X]/(X^N + 1),$$

and then rounds its coefficients to the nearest integers to obtain $M \in R$. M is then encrypted into an RLWE ciphertext. Provided that the centered decryption result is $M + e$, decoding returns

$$\frac{\tau(M+e)}{\Delta} = \mathbf{v} + \frac{\tau(M - M' + e)}{\Delta},$$

where $M - M'$ is the encoding error and e is the encryption noise. We refer to the length-$N/2$ vector $\tau(M+e)/\Delta$ as the *full slot vector* of the ciphertext.

The automorphism group of R consists of $\varphi_g : X \mapsto X^g$ for $g \in \mathbb{Z}_{2N}^*$. Applying φ_{5^i} to an R-element cyclically rotates its full slot vector upward by i positions, and applying φ_{-1} corresponds to slot-wise conjugation. Therefore, we denote $\texttt{Rot}_i = \varphi_{5^i}$ and $\texttt{Conj} = \varphi_{-1}$.

For cleaner noise analysis, we isolate the noise-free plaintext and rewrite $\mathrm{RLWE}_{s,q}(M; e)$ as $\mathrm{RLWE}_{s,q}(M'; e' = e + M - M')$.

Homomorphic operations. Because the canonical embedding is a ring homomorphism, polynomial addition and multiplication correspond to component-wise addition and multiplication of the full slot vectors. For $i = 0, 1$, we define two CKKS ciphertexts and their corresponding plaintexts as $c_i = (b_i, a_i) = \mathrm{RLWE}_{s,q}(M_i'; e_i')$ and $M_i' = \tau^{-1}(\Delta \mathbf{v}_i)$. The supported homomorphic operations are summarized as follows.

- $\texttt{CKKS.HomAdd}(c_0, c_1) = \mathrm{RLWE}_{s,q}(M_0' + M_1'; e_0' + e_1')$, which is a simple sum of the inputs. The two input ciphertexts are required to have the same scaling factor to produce a meaningful decryption result.
- $\texttt{CKKS.Rescale}(c_0, p \mid q) = \mathrm{RLWE}_{s,q/p}(M_0'/p; e_0'/p + e_{\mathrm{ms}})$, with e_{ms} the additional noise introduced by this step. The output is the value $(\lfloor b_0/p \rceil, \lfloor a_0/p \rceil)$ obtained by coefficient-wise division and rounding. The scaling factor of the plaintext polynomial is dropped from Δ to Δ/p.
- $\texttt{CKKS.KeySwitch}(c_0, \mathrm{KSK}_{s \to s'}) = \mathrm{RLWE}_{s',q}(M_0'; e_0' + e_{\mathrm{ks}})$, with e_{ks} the additional noise introduced by this step. e_{ks} is the sum of an e_{ms} and a small-coefficient linear combination of multiple encryption noises from $\mathrm{KSK}_{s \to s'}$.
- $\texttt{CKKS.HomRot}(c_0, i, \mathrm{KSK}_{\texttt{Rot}_i(s) \to s}) = \mathrm{RLWE}_{s,q}(\texttt{Rot}_i(M_0'); \texttt{Rot}_i(e_0') + e_{\mathrm{ks}})$. Key-switching is used to convert the decryption key from $\texttt{Rot}_i(s)$ back to s.

– $\texttt{CKKS.HomConj}(c_0, \mathrm{KSK}_{\texttt{Conj}(s)\to s}) = \mathrm{RLWE}_{s,q}(\texttt{Conj}(M_0'); \texttt{Conj}(e_0') + e_{\mathrm{ks}})$. This is similar to `CKKS.HomRot`.
– $\texttt{CKKS.HomPMult}(c_0, \lfloor M_1' \rceil) = \mathrm{RLWE}_{s,q}(M_0'M_1'; e_0'\lfloor M_1' \rceil + M_0'(\lfloor M_1' \rceil - M_1'))$, which is a component-wise multiplication of c_0 by $\lfloor M_1' \rceil$. The output ciphertext has scale Δ^2.
– $\texttt{CKKS.HomCMult}(c_0, c_1, \mathrm{KSK}_{s^2\to s}) = \mathrm{RLWE}_{s,q}(M_0'M_1'; e_0'M_1' + e_1'M_0' + e_0'e_1' + e_{\mathrm{ks}})$. This operation first tensors the two ciphertexts to produce a degree-two ciphertext under the secret $(1, s, s^2)$. A follow-up key-switching removes the s^2 term to restore the standard RLWE decryption format, and is therefore called relinearization. The output ciphertext has scale Δ^2.

2.4 Lattices and Lattice Reduction

Let $\mathbf{B} = [\mathbf{b}_0 \mid \cdots \mid \mathbf{b}_{n-1}] \in \mathbb{R}^{m\times n}$ have full column rank, where $n \le m$. The lattice generated by $\mathbf{B}$ is $\mathcal{L}(\mathbf{B}) = \{\mathbf{Bz} : \mathbf{z} \in \mathbb{Z}^n\} = \left\{\sum_{i\in[n]} z_i\mathbf{b}_i : z_i \in \mathbb{Z}\right\}$. It has rank n and lies in the n-dimensional subspace $\mathrm{span}(\mathbf{B}) \subseteq \mathbb{R}^m$. The columns $\mathbf{b}_i$ form a basis of the lattice. Its volume is $\mathrm{vol}(\mathcal{L}(\mathbf{B})) = \sqrt{\det(\mathbf{B}^T\mathbf{B})}$.

Gram–Schmidt orthogonalization. The Gram–Schmidt orthogonalization (GSO) of $\mathbf{B}$ consists of orthogonal vectors $\mathbf{b}_0^*, \ldots, \mathbf{b}_{n-1}^*$, obtained successively by projecting each $\mathbf{b}_i$ onto $\mathrm{span}(\mathbf{b}_0, \ldots, \mathbf{b}_{i-1})^\perp$, starting from $i = 0$. The lattice volume satisfies $\mathrm{vol}(\mathcal{L}(\mathbf{B})) = \prod_{i\in[n]} \|\mathbf{b}_i^*\|_2$.

Babai's nearest-plane algorithm. Given a target $\mathbf{t} \in \mathbb{R}^m$, the closest vector problem (CVP) asks for a vector in $\mathcal{L}(\mathbf{B})$ minimizing its Euclidean distance to $\mathbf{t}$. Babai's nearest-plane algorithm efficiently obtains an approximate solution using a basis and its GSO. Starting with $\mathbf{r} \leftarrow \mathbf{t}$, it processes the basis vectors in reverse order and computes

$$z_i \leftarrow \left\lfloor \frac{\langle \mathbf{r}, \mathbf{b}_i^* \rangle}{\|\mathbf{b}_i^*\|_2^2} \right\rceil, \qquad \mathbf{r} \leftarrow \mathbf{r} - z_i\mathbf{b}_i, \qquad i = n-1, \ldots, 0.$$

It returns $\mathbf{Bz} = \mathbf{t} - \mathbf{r} \in \mathcal{L}(\mathbf{B})$ as the approximate closest vector. This procedure is also referred to as *Babai rounding*.

For a target in $\mathrm{span}(\mathbf{B})$, the residual can be written as $\mathbf{r} = \sum_{i\in[n]} \varepsilon_i\mathbf{b}_i^*$ with $|\varepsilon_i| \le 1/2$. Therefore, the approximation quality depends strongly on the GSO lengths, which motivates applying BKZ before Babai rounding.

BKZ reduction and the Geometric Series Assumption. Lattice reduction replaces a basis with a better basis of the same lattice. The block Korkine–Zolotarev (BKZ) algorithm applies short-vector reduction to projected blocks of at most β consecutive basis vectors, where β is the block size [54]. Larger β generally yields shorter, more orthogonal basis vectors at a higher preprocessing cost. We denote a BKZ-β reduced basis of $\mathcal{L}(\mathbf{B})$ by $\mathbf{B}^{(\beta)}$.

The Geometric Series Assumption (GSA) heuristically models the lengths of the GSO vectors of a BKZ-reduced basis as a geometric progression [55].

Assumption 1 (Geometric Series Assumption (GSA)). *Consider a rank-n basis reduced by BKZ with block size β. Its GSO vectors satisfy*

$$\|\mathbf{b}_i^*\|_2 \approx \delta_\beta^{n-1-2i} \operatorname{vol}(\mathcal{L}(\mathbf{B}))^{1/n}, \qquad i \in [n],$$

where $\delta_\beta > 1$ is the root-Hermite factor of a BKZ-β reduced basis.

Following the standard BKZ estimate [20], we use $\delta_\beta \approx \left(\frac{\beta}{2\pi e}(\pi\beta)^{1/\beta}\right)^{1/(2(\beta-1))}$.

3 Plaintext Encoding from Cyclotomic Rounding

This section describes the plaintext encoding algorithms of GCKKS.

For $k \geq 4$, a complex vector $\mathbf{v} \in \mathbb{C}^{2N/k}$ is encoded into a polynomial in $R = \mathbb{Z}[X]/(X^N + 1)$. The whole encoding process is illustrated below[4].

$$\begin{array}{ccccccccc} \mathbb{C}^{2N/k} & \xrightarrow[\sim]{\mathtt{iDFT}_{N,k}} & \mathbb{C}[X]/(X^{2N/k}-\zeta_k) & \xrightarrow[\text{entrywise round}]{\mathtt{Round}_{N,k,w}} & \mathbb{Z}[\zeta_k][X]/(X^{2N/k}-\zeta_k) & \xrightarrow[\sim]{\mathtt{Expand}_{N,k}} & \mathbb{Z}[X]/(X^N+1) \\ \Delta\mathbf{v} & \mapsto & m' = \mathtt{iDFT}_{N,k}(\Delta\mathbf{v}) & \mapsto & m = \mathtt{Round}_{N,k,w}(m') & \mapsto & M = \mathtt{Expand}_{N,k}(m) \end{array}$$

The encoding consists of three maps

- The first isomorphism $\mathtt{iDFT}_{N,k}$ is defined by the inverse canonical embedding in $\mathbb{C}[X]/(X^{2N/k} - \zeta_k)$, whose roots are $\zeta_{2N}^{5^{i\cdot k/4}}$ for $i \in [2N/k]$.
- $\mathtt{Round}_{N,k,w}$ rounds each coefficient of m' to $\mathbb{Z}[\zeta_k]$ and is the core component in the encoding. The parameter w controls the encoding precision and will be detailed later.
- The last isomorphism $\mathtt{Expand}_{N,k}$ is defined by sending ζ_k to $X^{2N/k}$.

The $2N/k$ input slots $\mathbf{v}$ are evenly spaced in the $N/2$-sized full slot vector of M, where $\mathbf{v}[i]$ occupies the $(ik/4)$-th slot in M. These slots are called *useful slots*, while others are called *unused slots*. The scaling factor Δ is used to reduce the effect of rounding error on these useful slots.

The encoding quality is reflected by both the size of the useful-slot encoding error and the size of the unused slots, which are

$$B_{\text{use}} = \|\tau_{\text{useful}}(M) - \Delta\mathbf{v}\|_\infty \text{ and } \|M\|_{\text{can,unused}}.$$

The latter gives a lower bound on $\|M\|_{\text{can}}$, and we could set the scaling factor to $\Delta \approx \|M\|_{\text{can,unused}}$ without enlarging $\|M\|_{\text{can}}$. After encoding, the number of significant bits preserved in useful-slot values is $\log_2(\Delta/B_{\text{use}})$. The parameter w will control the encoding precision by adjusting both B_{use} and $\|M\|_{\text{can,unused}}$.

3.1 The Rounding from $\mathbb{C}$ to $\mathbb{Z}[\zeta_k]$

We study how to approximate $m'[i] \in \mathbb{C}$ in $\mathbb{Z}[\zeta_k]$ with an adjustable precision.

[4] When $k = 4$, the encoding degenerates to standard CKKS encoding, and $\mathtt{Round}_{N,k,w}$ rounds every coefficient in m' to its nearest Gaussian integer.

Rounding through CVP. We want to round $u \in \mathbb{C}$ into $v = \sum_{i \in [k/2]} c_i \cdot \zeta_k^i$, where $c_i \in \mathbb{Z}$. The coefficients c_i should be small in order to control the size of $\|M\|_{\text{can,unused}}$. The intuitive idea is to reduce the task to solving the following CVP instance

$$\mathbf{B}_{k,w} = \begin{pmatrix} \mathrm{Re}(\zeta_k^0) & \cdots & \mathrm{Re}(\zeta_k^{k/2-1}) \\ \mathrm{Im}(\zeta_k^0) & \cdots & \mathrm{Im}(\zeta_k^{k/2-1}) \\ w & & \\ & \ddots & \\ & & w \end{pmatrix}, \qquad \mathbf{B}_{k,w} \begin{pmatrix} c_0 \\ \vdots \\ c_{k/2-1} \end{pmatrix} \approx \mathbf{t} = \begin{pmatrix} \mathrm{Re}(u) \\ \mathrm{Im}(u) \\ 0 \\ \vdots \\ 0 \end{pmatrix},$$

where w controls whether the solution favors smaller rounding error or smaller c_i.

However, $\mathbf{t}$ does not belong to $\mathrm{span}(\mathbf{B}_{k,w})$. In fact, its orthogonal component has a nonzero ℓ_2-norm of $\Theta(\frac{w}{\sqrt{k}}|u|)$ for small values of w, which causes additional approximation error.

The optimal CVP target. Finding the right CVP target is not straightforward when rounding a single complex scalar, but it becomes natural when we look for the *optimal* unrounded encoded polynomial $M' \in \mathbb{R}[X]/(X^N+1)$. Intuitively, M' should exactly preserve the useful slots while having zero-filled unused slots, i.e.,

$$\tau_{\text{useful}}(M') = \Delta\mathbf{v}, \qquad \tau_{\text{unused}}(M') = \mathbf{0}.$$

The rounding procedure then consists of the following steps

- Calculate M' from m' using a CRT reconstruction map

  $$\texttt{Lift}_{N,k} : \mathbb{C}[X]/(X^{2N/k} - \zeta_k) \to \mathbb{R}[X]/(X^N+1), \qquad m' \mapsto M',$$

  where M' satisfies

  $$M' \equiv m' \bmod X^{2N/k} - \zeta_k, \quad M' \equiv 0 \bmod \frac{X^{N/2} - J}{X^{2N/k} - \zeta_k}.$$

- For $i \in [2N/k]$, the CVP target for rounding $m'[i]$ is constructed as

  $$\mathbf{t}_i = \big(\mathrm{Re}(m'[i]) \;\; \mathrm{Im}(m'[i]) \;\; wM'[i] \;\; wM'[i{+}2N/k] \;\; \cdots \;\; wM'[i{+}N{-}2N/k]\big)^T.$$

  Then, we solve CVP in $\mathcal{L}(\mathbf{B}_{k,w})$ with a target of $\mathbf{t}_i$, getting a solution of

  $$\mathbf{B}_{k,w} \cdot \big(c_{i,0}, c_{i,1}, \cdots, c_{i,k/2-1}\big)^T = \big(u_i, v_i, \cdots\big)^T,$$

  where $u_i + v_i \cdot J$ is close to $m'[i]$.
  Collecting the results for all i leads to

  $$m = \sum_{i \in [2N/k]} \big(\sum_{j \in [k/2]} c_{i,j} \cdot \zeta_k^j \big) X^i.$$

– The encoded integer polynomial M is constructed as $M = \texttt{Expand}_{N,k}(m)$.

Achieving small encoding error in the useful slots requires $\mathbf{t}_i \in \mathrm{span}(\mathbf{B}_{k,w})$. This is true because $M' \equiv m' \bmod X^{2N/k} - \zeta_k$, implying

$$m'[i] = \sum_{j \in [k/2]} M'[i + j \cdot 2N/k] \cdot \zeta_k^j.$$

Therefore

$$\mathbf{t}_i = \mathbf{B}_{k,w} \cdot (M'[i] \;\; M'[i + 2N/k] \;\; M'[i + 4N/k] \;\; \cdots)^T \in \mathrm{span}(\mathbf{B}_{k,w}).$$

Lattice preprocessing. Solving CVP during plaintext encoding is costly and should be accelerated through lattice preprocessing. We therefore preprocess $\mathbf{B}_{k,w}$ into a BKZ-reduced basis $\mathbf{B}_{k,w}^{(\beta)}$. In plaintext encoding, approximate CVPs are solved by Babai rounding on $\mathbf{B}_{k,w}^{(\beta)}$.

During preprocessing, we record the unimodular matrix $\mathbf{U} \in \mathbb{Z}^{k/2 \times k/2}$ satisfying $\mathbf{B}_{k,w}^{(\beta)} = \mathbf{B}_{k,w}\mathbf{U}$, together with the GSO of $\mathbf{B}_{k,w}^{(\beta)}$. These data depend only on (k, w, β) and can be reused across encodings. Let $\texttt{BabaiCoeff}(\mathbf{B}, \mathbf{t})$ denote the integer coefficient vector returned by Babai's nearest-plane algorithm on basis $\mathbf{B}$ and target $\mathbf{t}$.

The full encoding algorithm is presented in Algorithm 1. Since Babai rounding has quadratic complexity in the lattice dimension, the encoding algorithm has complexity $O(Nk)$.

Algorithm 1 $\texttt{LatticeEncode}_{N,k,w,\beta,\Delta}(\mathbf{v})$

Require: Slot vector $\mathbf{v} \in \mathbb{C}^{2N/k}$, scale Δ, parameters (N, k, w), and preprocessed $\mathbf{B}_{k,w}^{(\beta)} = \mathbf{B}_{k,w}\mathbf{U}$ with its GSO
Ensure: Encoded polynomial $M \in R$

$m' \leftarrow \texttt{iDFT}_{N,k}(\Delta \mathbf{v})$
$M' \leftarrow \texttt{Lift}_{N,k}(m')$
$m \leftarrow 0$
for $i \leftarrow 0$ **to** $2N/k - 1$ **do**
  $\mathbf{t}_i \leftarrow \big(\mathrm{Re}(m'[i]), \mathrm{Im}(m'[i]), wM'[i], wM'[i + 2N/k], \ldots, wM'[i + (k/2 - 1) \cdot 2N/k]\big)^T$
  $\mathbf{c}_{i,\mathrm{BKZ}} \leftarrow \texttt{BabaiCoeff}(\mathbf{B}_{k,w}^{(\beta)}, \mathbf{t}_i)$
  $\mathbf{c}_i \leftarrow \mathbf{U}\mathbf{c}_{i,\mathrm{BKZ}}$ ▷ Converts $\mathbf{B}_{k,w}^{(\beta)}$-coordinates into $\mathbf{B}_{k,w}$-coordinates
  $m \leftarrow m + \big(\sum_{j \in [k/2]} \mathbf{c}_i[j] \zeta_k^j\big) X^i$
end for
return $\texttt{Expand}_{N,k}(m)$ ▷ $M[i + j \cdot 2N/k] = \mathbf{c}_i[j]$

3.2 Noise Analysis

We give conclusions about the size of the useful-slot encoding errors and unused-slot values in M. Our analysis is based on the following assumption.

Assumption 2. *Denote by* $\mathbf{r}_i = \sum_{j\in[k/2]} \varepsilon_{i,j}\mathbf{b}_j^*$ *the Babai rounding error for* $i \in [2N/k]$. *Then* $\varepsilon_{i,j}$ *are independent and uniformly distributed on* $[-1/2, 1/2]$.

The main conclusion is given as follows.

Proposition 1 (Babai-based encoding quality). *Under Assumption 2 and Assumption 1, the useful-slot encoding errors and unused-slot values follow* $\mathcal{CN}(0, V_{\text{use}})$ *and* $\mathcal{CN}(0, V_{\text{unused}})$, *where*

$$V_{\text{use}} \lessapprox \frac{N}{24}(1+\delta_\beta^{-4})\delta_\beta^{k-2}(w^2+k/4)^{4/k-1}w^{2-8/k},$$
$$V_{\text{unused}} \lessapprox \frac{N(w^2+k/4)^{4/k}w^{-8/k}}{12}\left(\sum_{j\in[k/2]}\delta_\beta^{k-2-4j} - \alpha(\delta_\beta^{-k+6}+\delta_\beta^{-k+2})\right).$$

For fixed (N, k, β) and $w \to 0$, the useful-slot and unused-slot standard deviations are respectively $O(w^{1-4/k})$ and $O(w^{-4/k})$. Under the common setting of $\|\mathbf{v}\|_\infty \le 1$, we can set the scaling factor Δ to $\|M\|_{\text{can,unused}}$ so that $\Delta = \|M\|_{\text{can}} = O(w^{-4/k})$. Under this setting, the number of significant bits in $\mathbf{v}$ preserved by encoding is about

$$\log_2(\sqrt{V_{\text{unused}}/V_{\text{use}}}) = \Theta(-\log_2(w)).$$

With $w = 2^{-p}$, this means that we can encode complex values with p-bit precision and a canonical norm of $O(2^{4p/k})$.

Preparing for the proof. We first partition $\mathbf{B}_{k,w}$ as

$$\mathbf{B}_{k,w} = \begin{pmatrix}\mathbf{h}_0^T\\ \mathbf{h}_1^T\\ w\mathbf{I}\end{pmatrix},$$

where

$$\mathbf{h}_0[i] = \cos(2\pi i/k), \qquad \mathbf{h}_1[i] = \sin(2\pi i/k), \quad i \in [k/2].$$

Lemma 1. $\text{vol}(\mathcal{L}(\mathbf{B}_{k,w})) = (w^2 + \frac{k}{4}) \cdot w^{k/2-2}$.

Proof. According to our partition of $\mathbf{B}_{k,w}$,

$$\text{vol}(\mathcal{L}(\mathbf{B}_{k,w})) = \det(\mathbf{B}_{k,w}^T\mathbf{B}_{k,w})^{1/2} = \det(\mathbf{h}_0\mathbf{h}_0^T + \mathbf{h}_1\mathbf{h}_1^T + w^2\mathbf{I})^{1/2}.$$

Observe that $\langle \mathbf{h}_0, \mathbf{h}_1\rangle = 0$. This means that $\mathbf{h}_0\mathbf{h}_0^T + \mathbf{h}_1\mathbf{h}_1^T$ has rank two, and its only two nonzero eigenvalues are given by

$$(\mathbf{h}_0\mathbf{h}_0^T + \mathbf{h}_1\mathbf{h}_1^T)\mathbf{h}_i = \|\mathbf{h}_i\|^2\mathbf{h}_i = \frac{k}{4}\mathbf{h}_i, \quad i = 0, 1.$$

Therefore, $\mathbf{B}_{k,w}^T\mathbf{B}_{k,w}$ has two eigenvalues of $w^2 + k/4$, and $k/2 - 2$ eigenvalues of w^2. Then

$$\mathrm{vol}(\mathcal{L}(\mathbf{B}_{k,w})) = (w^2 + \frac{k}{4}) \cdot w^{k/2-2}. \qquad \square$$

Under GSA, the ℓ_2-norm of $\mathbf{b}_i^*$ can be estimated as

$$\|\mathbf{b}_i^*\| \approx \delta_\beta^{k/2-1-2i}(w^2 + k/4)^{2/k} \cdot w^{1-4/k}.$$

Lemma 2. $\mathrm{span}(\mathbf{B}_{k,w})^\perp$ *is spanned by two orthogonal vectors*

$$\mathbf{x}_0 = \begin{pmatrix} -w \\ 0 \\ \mathbf{h}_0 \end{pmatrix} \quad \textit{and} \quad \mathbf{x}_1 = \begin{pmatrix} 0 \\ -w \\ \mathbf{h}_1 \end{pmatrix},$$

satisfying $\|\mathbf{x}_0\| = \|\mathbf{x}_1\| = \sqrt{w^2 + k/4}$.

Proof. $\mathrm{span}(\mathbf{B}_{k,w})^\perp$ has a dimension of two since $\mathbf{B}_{k,w}$ is full rank. Direct calculation gives $\langle \mathbf{x}_0, \mathbf{x}_1 \rangle = 0$, $\mathbf{B}_{k,w}^T\mathbf{x}_i = \mathbf{0}$ for $i = 0, 1$, and the values of $\|\mathbf{x}_i\|$. $\square$

The following result will also be useful for our analysis.

Lemma 3 (Ky Fan's eigenvalues inequality [35]). *Let* $\mathbf{A} \in \mathbb{R}^{m\times m}$ *be symmetric and positive semidefinite, with eigenvalues* $\lambda_0 \geq \lambda_1 \geq \cdots \geq \lambda_{m-1}$. *For any* d*-dimensional subspace* $V \subseteq \mathbb{R}^m$ *and its orthogonal projector* $\mathbf{P}_V$,

$$\sum_{i=m-d}^{m-1} \lambda_i \leq \mathrm{tr}(\mathbf{P}_V\mathbf{A}) \leq \sum_{i=0}^{d-1} \lambda_i.$$

Proof (Proposition 1). Denote the residual produced by Babai rounding in Algorithm 1 by

$$\mathbf{r}_i = \sum_{j\in[k/2]} \varepsilon_{i,j}\mathbf{b}_j^*.$$

According to the construction of the CVP instance,

$$\mathbf{B}_{k,w}\mathbf{c}_i = \mathbf{t}_i - \mathbf{r}_i.$$

The first two rows in the linear system above directly correspond to the rounding results of the real and imaginary parts of $m'[i]$. These two rows can be combined and rewritten as

$$(m' - m)[i] = \mathbf{r}_i[0] + \mathbf{r}_i[1]J.$$

The useful-slot encoding error, $\tau_{\mathrm{useful}}(M' - M)$, is exactly

$$\tau(m' - m) = \tau\big(\sum_{i\in[2N/k]} (\mathbf{r}_i[0] + \mathbf{r}_i[1]J)X^i\big).$$

Meanwhile, the remaining $k/2$ rows describe how far M is from M', i.e.,

$$(M' - M)[i + j \cdot 2N/k] = \mathbf{r}_i[j + 2]/w.$$

Since $\tau_{\text{unused}}(M') = \mathbf{0}$ by construction, we know $\tau_{\text{unused}}(M)$ is equal to

$$-w^{-1}\tau_{\text{unused}}\Big(\sum_{i\in[2N/k]}\sum_{j\in[k/2]}\mathbf{r}_i[j+2]X^{i+j\cdot 2N/k}\Big).$$

A projection-based analysis. The same residual $\mathbf{r}_i$ determines both distributions. By Assumption 2, $\varepsilon_{i,j}$ are independent, centered, and have variance $1/12$, so we have $\mathrm{E}(\mathbf{r}_i) = \mathbf{0}$ and

$$\mathbf{C} = \mathrm{E}(\mathbf{r}_i\mathbf{r}_i^T) = \frac{1}{12}\sum_{j\in[k/2]}\mathbf{b}_j^*(\mathbf{b}_j^*)^T.$$

Let $H = \mathrm{span}(\mathbf{B}_{k,w})$. The matrix $\mathbf{C}$ is supported on H and, because the GSO vectors are orthogonal, its eigenvalues on H are $\|\mathbf{b}_j^*\|^2/12$.

To separate the first two coordinates of $\mathbf{r}_i$ from the remaining ones, let

$$\mathbf{e}_t^{\parallel} = \mathrm{Proj}_H(\mathbf{e}_t) = \mathbf{e}_t - \frac{w}{w^2+k/4}\mathbf{x}_t, \qquad t = 0, 1.$$

By the preceding orientation lemma, these two projections are orthogonal and have the same squared norm $\alpha = \frac{k/4}{w^2+k/4}$. Set $V = \mathrm{span}(\mathbf{e}_0^{\parallel}, \mathbf{e}_1^{\parallel})$ and let $\mathbf{P}_V$ be the orthogonal projector onto V. Since $\mathbf{r}_i \in H$, we have

$$\mathrm{E}(\mathbf{r}_i[0]^2 + \mathbf{r}_i[1]^2) = \mathrm{E}(\langle\mathbf{r}_i, \mathbf{e}_0^{\parallel}\rangle^2 + \langle\mathbf{r}_i, \mathbf{e}_1^{\parallel}\rangle^2) = \alpha\,\mathrm{tr}(\mathbf{P}_V\mathbf{C}). \tag{3}$$

Useful-slot encoding error distribution. We first compute the variance of $\mathbf{r}_i[0] + \mathbf{r}_i[1]J$, before using the Central Limit Theorem to model the entries of $\tau(m' - m)$ as centered Gaussian distributions. By (3), Lemma 3, GSA, and the eigenvalues of $\mathbf{C}$,

$$\mathrm{Var}(\mathbf{r}_i[0] + \mathbf{r}_i[1]J) = \alpha\,\mathrm{tr}(\mathbf{P}_V\mathbf{C}) \le \frac{\alpha}{12}(\|\mathbf{b}_0^*\|^2 + \|\mathbf{b}_1^*\|^2)$$

Each entry in $\tau(m'-m)$ is the inner product of the coefficient errors and a vector of roots of unity, so its variance is $2N/k$ times $\mathrm{Var}(\mathbf{r}_i[0] + \mathbf{r}_i[1]J)$.

Unused-slot value distribution. The last $k/2$ coordinates of $\mathbf{r}_i$ come from the same Babai rounding and may be correlated. Therefore, instead of coordinate-wise analysis, we bound their total second moment $\mathrm{E}(\|\mathbf{r}_i[2:]\|^2)$, which is

$$\mathrm{E}(\|\mathbf{r}_i[2:]\|^2) = \mathrm{E}(\|\mathbf{r}_i\|^2) - \mathrm{E}(\mathbf{r}_i[0]^2 + \mathbf{r}_i[1]^2) = \mathrm{tr}(\mathbf{C}) - \alpha\,\mathrm{tr}(\mathbf{P}_V\mathbf{C}). \tag{4}$$

To upper-bound this energy, it remains to find a lower bound on $\mathrm{tr}(\mathbf{P}_V\mathbf{C})$, which is given by Lemma 3 as

$$\mathrm{tr}(\mathbf{P}_V\mathbf{C}) \ge \frac{1}{12}\left(\|\mathbf{b}_{k/2-2}^*\|^2 + \|\mathbf{b}_{k/2-1}^*\|^2\right).$$

It follows that

$$\mathrm{E}(\|\mathbf{r}_i[2:]\|^2) \le \frac{1}{12}\left(\sum_{j\in[k/2]}\|\mathbf{b}_j^*\|^2 - \alpha(\|\mathbf{b}_{k/2-2}^*\|^2 + \|\mathbf{b}_{k/2-1}^*\|^2)\right).$$

Finally, fix an unused canonical-embedding point ξ. From the expression for $\tau_{\text{unused}}(M)$ above, its entry at ξ is

$$M(\xi) = -\frac{1}{w} \sum_{i \in [2N/k]} \xi^i \sum_{j \in [k/2]} \mathbf{r}_i[j+2] \xi^{j \cdot 2N/k}. \tag{5}$$

The outer summands are independent and centered by Assumption 2. Since each row of a $k/2$-sized canonical embedding has norm $\sqrt{k/2}$, the inner sum is bounded by $\sqrt{k/2} \cdot \|\mathbf{r}_i[2:]\|$. The variance of $M(\xi)$ is at most

$$w^{-2} \frac{2N}{k} \cdot \frac{k}{2} \operatorname{E}(\|\mathbf{r}_i[2:]\|^2) = \frac{N}{w^2} \operatorname{E}(\|\mathbf{r}_i[2:]\|^2).$$

Combining this with the preceding estimate gives the claimed upper bound on V_{unused}. Applying the Central Limit Theorem to the sum over i gives the claimed Gaussian model for each unused-slot value. □

As indicated by the proof, when the GSO vectors are unknown, looseness arises from the worst-case GSO-orientation and canonical-embedding bounds. Once the GSO vectors are determined, tighter bounds can be calculated using the exact traces in (3) and (4) and the exact canonical-embedding values in (5).

3.3 A Simpler Encoding from Polynomial Multiplication

We present a simplified encoding process that will play an important role later in Section 4. As the core observation, a polynomial $F \in \mathbb{Z}[X^{2N/k}]/(X^N+1)$ can be constructed from any short vector $\mathbf{b} \in \mathcal{L}(\mathbf{B}_{k,w})$[5] as

$$F = w^{-1} \sum_{j \in [k/2]} \mathbf{b}[j+2] X^{j \cdot 2N/k}, \tag{6}$$

whose useful slots contain the *same* complex value

$$c_m = \mathbf{b}[0] + \mathbf{b}[1] J.$$

An analysis similar to Proposition 1 then gives $|c_m| = O(w^{1-4/k})$ and $|\tau_{\text{unused}}(F)| = O(w^{-4/k})$. Therefore, *multiplication by F can be used to adjust the magnitude of useful- and unused-slot values, thereby converting a standard CKKS-encoded plaintext into a GCKKS plaintext.*

Based on this observation, the encoding process using F becomes

$$\begin{array}{ccccccccc} \mathbb{C}^{2N/k} & \xrightarrow{\texttt{iDFT}_{N,k}} & \mathbb{C}[X]/(X^{2N/k}-\zeta_k) & \xrightarrow{\texttt{Lift}_{N,k}} & \mathbb{R}[X]/(X^N+1) & \xrightarrow{\text{round}} & \mathbb{Z}[X]/(X^N+1) & \xrightarrow{\times F} & \mathbb{Z}[X]/(X^N+1) \\ \Delta c_m^{-1}\mathbf{v} & \overset{\sim}{\mapsto} & m' = \texttt{iDFT}_{N,k}(\Delta c_m^{-1}\mathbf{v}) & \mapsto & M' = \texttt{Lift}_{N,k}(m') & \mapsto & M = \lfloor M' \rceil & \mapsto & M \cdot F \end{array}$$

We denote this encoder by $\texttt{MultEncode}_{N,k,\Delta,F}$. More precisely,

$$\texttt{MultEncode}_{N,k,\Delta,F}(\mathbf{v}) = \big\lfloor \texttt{Lift}_{N,k}\big(\texttt{iDFT}_{N,k}(\Delta c_m^{-1}\mathbf{v})\big) \big\rceil F.$$

[5] One may take $\mathbf{b} = \mathbf{b}_0^*$, the first vector in $\mathbf{B}_{k,w}^{(\beta)}$.

This reduces the encoding time complexity from $O(Nk)$ to $O(N \log N)$.

To see why the encoding is correct, observe

$$\tau_{\text{useful}}(MF) = \tau_{\text{useful}}(M' + (M - M')) \odot \tau_{\text{useful}}(F) = \Delta\mathbf{v} + c_m \cdot \tau_{\text{useful}}(M - M'),$$

where $\|\tau_{\text{useful}}(M - M')\|_\infty \le N/2$ due to $\|M - M'\|_\infty \le 1/2$, so the encoding error still has a standard deviation of $O(w^{1-4/k})$. For the unused slots,

$$\tau_{\text{unused}}(MF) = \tau_{\text{unused}}(M - M') \odot \tau_{\text{unused}}(F)$$

also has a standard deviation of $O(w^{-4/k})$.

Lemma 4 (Canonical embedding of F). *Suppose F is constructed from $\mathbf{b} \in \mathcal{L}(\mathbf{B}_{k,w})$ as in (6). At every useful embedding point, F evaluates to c_m, where*

$$|c_m| \le \sqrt{\frac{k/4}{w^2 + k/4}} \|\mathbf{b}\|.$$

At every unused canonical-embedding point ξ,

$$|F(\xi)| \le \sqrt{\frac{k}{2w^2}} \|\mathbf{b}\|.$$

Proof. The projection calculation in the proof of Proposition 1 directly gives

$$|c_m|^2 = \frac{k/4}{w^2 + k/4} \|\mathbf{P}_V \mathbf{b}\|^2 \le \frac{k/4}{w^2 + k/4} \|\mathbf{b}\|^2$$
$$|F(\xi)|^2 \le \frac{k}{2w^2} \|\mathbf{b}[2:]\|^2 = \frac{k}{2w^2} (\|\mathbf{b}\|^2 - |c_m|^2) \le \frac{k}{2w^2} \|\mathbf{b}\|^2. \qquad \square$$

emp

The variances of the useful-slot encoding error and the unused-slot values of MF directly follow from the following assumption.

Assumption 3. *Each entry in $\tau(M - M')$ follows $\mathcal{CN}(0, N/12)$.*

Corollary 1 (Encoding quality of `MultEncode`). *Under Assumption 3, the useful-slot encoding errors and unused-slot values of MF follow $\mathcal{CN}(0, V_{\text{use}})$ and $\mathcal{CN}(0, V_{\text{unused}})$ with*

$$V_{\text{use}} \le \frac{k/4}{w^2 + k/4} \cdot \frac{N}{12} \|\mathbf{b}\|^2, \qquad V_{\text{unused}} \le \frac{Nk}{24w^2} \left(\|\mathbf{b}\|^2\right).$$

Specifically, we may take $\mathbf{b} = \mathbf{b}_0^*$. Then Assumption 1 gives

$$V_{\text{use}} \lessapprox \frac{Nk}{48} \delta_\beta^{k-2} (w^2 + k/4)^{4/k-1} w^{2-8/k},$$
$$V_{\text{unused}} \lessapprox \frac{Nk}{24} \delta_\beta^{k-2} (w^2 + k/4)^{4/k} w^{-8/k}.$$

3.4 Empirical Verification

We conducted experiments to assess the preceding bounds and distributional assumptions. For Proposition 1, we tested the centeredness, circularity, and tail probabilities of the noise distributions. For Lemma 4 and Assumption 3, we tested $\|F\|_{\text{can,useful}}$ and $\|F\|_{\text{can,unused}}$, as well as the centeredness, circularity, and tail probabilities of $\tau(M - M')$.

For variance and canonical bound verification, the tighter bounds calculated from the GSO vectors are taken as the reference values. The tests support the theoretical analysis, with the results detailed in Appendix C due to limited space. The looseness of these upper bounds is also examined in Appendix D.

4 Homomorphic Operations in GCKKS

In the previous section, we constructed a plaintext encoding algorithm that encodes $2N/k$ complex values into a polynomial with a canonical norm of $O(w^{-4/k})$, while preserving $\log_2(w^{-1})$ bits of precision. However, moving from plaintexts to ciphertexts introduces an additional challenge:

Encryption and modulus switching introduce extra noise into the plaintext. It has a standard deviation of $\Omega(\sqrt{N})$ in every slot, which ruins the precision in useful slots.

4.1 A New Decryption Format for Automatic Noise Rebalancing

Our solution to the problem above is to extend the mult-by-F encoding structure in Section 3.3 to ciphertexts. Inside the slots, this has the immediate effect of rebalancing any noise introduced by homomorphic operations, in exactly the same manner as the rounding error is rebalanced in the mult-by-F encoding.

Reusing the notation from Section 3.3, the modified encryption and decryption formats are as follows.

The encoded polynomial is M, satisfying $\tau_{\text{useful}}(M) = \Delta c_m^{-1}\mathbf{v} + \tau_{\text{useful}}(M - M')$. M is encrypted in the standard RLWE format, i.e., $c = (b = -as + e + M, a) \in R_q^2$, and its decryption is augmented by multiplication by F, producing

$$\mathtt{Dec}_{s,F}(c) \equiv (b + as)F \equiv (M + e)F \bmod q.$$

We call this augmented decryption format *F-decryption* and call F the *decryption factor*.

One might worry that decryption may give incorrect results when M wraps around q. In fact, decryption is correct as long as $\|(M+e)F\|_\infty < q/2$, in which case any q-overflow in M disappears during decryption. This phenomenon in homomorphic computation is known as *pseudo overflow* [5].

The stored RLWE pair together with its decryption factor is denoted by $\text{RLWE}_{s,q,F}(M; e)$. The decrypted polynomial then satisfies

$$\begin{aligned}\tau_{\text{useful}}((M+e)F) &= \Delta\mathbf{v} + \tau_{\text{useful}}(e + M - M') \odot \tau_{\text{useful}}(F)\\ \tau_{\text{unused}}((M+e)F) &= \tau_{\text{unused}}(e + M - M') \odot \tau_{\text{unused}}(F).\end{aligned}$$

According to Lemma 4, both the rounding error $M - M'$ and RLWE error e have their useful-slot values reduced to magnitude $O(w^{1-4/k})$ and their unused-slot values increased to $O(w^{-4/k})$.

Other homomorphic operations, including rescaling, also take place on (b, a) instead of (bF, aF), so their noise is similarly rebalanced by F-decryption[6].

4.2 Homomorphic Operations and Noise Analysis

We describe the homomorphic operations under the modified decryption format.

The F-decryption format creates a link between a pair of CKKS and GCKKS ciphertexts. Namely, a GCKKS ciphertext with scale Δ, when viewed under the standard CKKS decryption format, becomes a standard CKKS ciphertext with scale $\Delta/|c_m|$ (up to pseudo overflows in q).

This link between CKKS and GCKKS ciphertexts reduces the implementations of most GCKKS operations to their standard CKKS counterparts. The only exceptions are ciphertext multiplication and conjugation, because they temporarily break the mult-by-F decryption format and use extra operations to restore it.

The homomorphic operations introduce noise in the standard CKKS domain, which is rebalanced by F-decryption in the GCKKS domain. Therefore, the link also reduces the relative noise growth in the useful slots of GCKKS to that of CKKS. This is given by Theorem 1, which serves as a correctness proof of the GCKKS homomorphic operations.

Homomorphic operations. For $i = 0, 1$, define two GCKKS ciphertexts as

$$c_i = (b_i, a_i) = \mathrm{RLWE}_{s,q,F}(M_i; e_i), \text{ where } M_i = \lfloor \mathtt{Lift}_{N,k}(\mathtt{iDFT}_{N,k}(\Delta c_m^{-1} \mathbf{v}_i)) \rceil.$$

- $\mathtt{GCKKS.HomAdd}(c_0, c_1) = \mathtt{CKKS.HomAdd}(c_0, c_1)$. This operation requires the two ciphertexts to have identical Δ and F.
- $\mathtt{GCKKS.Rescale}(c_0, q') = \mathtt{CKKS.Rescale}(c_0, q')$.
- $\mathtt{GCKKS.HomPMult}(c_0, M_1 F) = \mathtt{CKKS.HomPMult}(c_0, M_1 F)$.
- $\mathtt{GCKKS.KeySwitch}(c_0, \mathrm{KSK}_{s \to s'}) = \mathtt{CKKS.KeySwitch}(c_0, \mathrm{KSK}_{s \to s'})$.
- $\mathtt{GCKKS.HomRot}(c_0, i, \mathrm{KSK}_{\mathtt{Rot}_{ik/4}(s) \to s}) = \mathtt{CKKS.HomRot}(c_0, ik/4, \mathrm{KSK}_{\mathtt{Rot}_{ik/4}(s) \to s})$. The i-th useful slot occupies the $i \cdot k/4$-th slot in the full CKKS slot vector, so a GCKKS rotation of offset i can be implemented using a CKKS rotation of offset $i \cdot k/4$. Even better, every valid GCKKS rotation automorphism preserves the F-decryption format, because $F \in \mathbb{Z}[X^{2N/k}]/(X^N + 1)$ remains invariant under $\mathtt{Rot}_{k/4}$.
- $\mathtt{GCKKS.HomConj}(c_0, \mathrm{KSK}_{\mathtt{Conj}(s) \to s})$ has the same key material as $\mathtt{CKKS.HomConj}$ but has a different computation.

[6] Interested readers may refer to Section F.2 for our first attempt to deal with modulus-switching noise by manually re-encoding it.

Applying $\texttt{Conj}$ to $c_0 = (b_0, a_0)$ simultaneously changes the decryption factor from F to $\texttt{Conj}(F)$. To restore the decryption factor, we first reinterpret $(\texttt{Conj}(b_0), \texttt{Conj}(a_0))$ as if it has a decryption factor of F, giving

$$\begin{aligned}\texttt{Dec}_{\texttt{Conj}(s),F}((\texttt{Conj}(b_0), \texttt{Conj}(a_0))) &= \big(\texttt{Conj}(b_0) + \texttt{Conj}(a_0)\texttt{Conj}(s)\big) \cdot F \\ &= \texttt{Conj}(M_0 + e_0)F,\end{aligned}$$

whose useful slots contain

$$\tau_{\text{useful}}(\texttt{Conj}(M_0 + e_0)) \odot \tau_{\text{useful}}(F) \approx \bar{c}_m^{-1} \Delta \bar{\mathbf{v}}_0 \cdot c_m.$$

Then, the extra factor of $c_m \bar{c}_m^{-1}$ is cancelled through one $\texttt{GCKKS.HomPMult}$ followed by a rescaling. The remaining key-switching is implemented using $\texttt{GCKKS.KeySwitch}$.

– $\texttt{GCKKS.HomCMult}(c_0, c_1, \text{KSK}_{s^2 \to s})$ shares the same key material and most of the computation as $\texttt{CKKS.HomCMult}$.
The tensoring stage has the side effect of squaring the decryption factor, so the post-tensoring ciphertext requires F^2-decryption. To restore the old F-decryption format, an extra F is multiplied by the stored RLWE components immediately after tensoring. The remaining relinearization process has a workflow identical to that in $\texttt{CKKS.HomCMult}$. We note that a double-CRT representation of F can be precomputed, so this extra mult-by-F does not involve any expensive NTT operations.

Theorem 1 (Useful-slot noise analysis). *A GCKKS ciphertext with scale Δ and decryption factor F has the same useful-slot relative noise growth as a CKKS ciphertext with scale $\Delta/|c_m|$, with c_m the useful-slot value of F.*

Proof. The growth of ciphertext noise comes from two sources. The first is the propagation of existing noises, during operations like homomorphic addition or the tensoring stage in homomorphic multiplication. The second is the new noises introduced by encryption and modulus/key switching.

The new noises are message-independent and have the same RLWE values whether the decryption format is in CKKS or GCKKS. Their slot values are additionally multiplied by c_m during GCKKS decryption. Therefore, relative to these newly introduced noises, a GCKKS ciphertext with scale Δ has the same relative useful-slot precision as a CKKS ciphertext with scale $\Delta/|c_m|$. The complex phase of c_m does not affect the analysis due to the common assumption that these noises have a circularly symmetric distribution inside each slot.

Finally, observe that the relative noise growth during noise propagation is irrelevant to the scale used. Therefore, the equivalent scale $\Delta/|c_m|$ determines the overall relative noise growth in GCKKS ciphertexts. □

Challenges in noise analysis. The useful-slot noise analysis directly reduces to that of CKKS, but the unused-slot noise analysis presents new challenges.

In standard CKKS, the slots have magnitudes $\approx \Delta$ due to the common assumption $\|\mathbf{v}\| \leq 1$. During homomorphic computation, the slot magnitudes

are determined by the circuit evaluated and the initial slot values $\mathbf{v}$. Regarding noise growth in homomorphic multiplication, the product of two slots $m_0 + e_0$ and $m_1 + e_1$ has noise dominated by $m_0e_1 + m_1e_0$, while the product e_0e_1 is negligible compared to the other terms.

However, the unused slots of CKKS consist purely of random noises, leading to multiple challenges. First, a sufficiently large Δ is required to stabilize the size of unused slots throughout homomorphic operations. Otherwise, they may diverge to infinity during homomorphic computation. Second, after homomorphic squarings, the unused slots contain high-order powers of noise, which are non-negligible and have much heavier tails than a Gaussian distribution. Therefore, we cannot adopt the Gaussian-based noise analyses of existing schemes and must resort to worst-case analysis.

Stabilizing the unused-slot magnitude. The idea of our noise analysis is to derive worst-case propagation formulas for the unused-slot magnitude in basic homomorphic circuits and choose a sufficiently large Δ to stabilize it in these circuits. The resulting noise bound then carries over to larger circuits composed of these basic circuits.

The first basic circuit is homomorphic multiplication, i.e., `GCKKS.HomCMult` followed by a `GCKKS.Rescale`. Suppose the unused slots of two ciphertexts, denoted by $\mathbf{t}_i$, have a common bound B, and the scaling factor is Δ. During a homomorphic multiplication, the unused-slot values become $\mathbf{t}_0 \odot \mathbf{t}_1/\Delta + e_{\mathrm{ks}}/\Delta + e_{\mathrm{ms}}$. Let B_{added} be an upper bound on the introduced noise terms $e_{\mathrm{ks}}/\Delta + e_{\mathrm{ms}}$. The ciphertext has a noise bound of

$$f_{\mathrm{mult}}(B) = B^2/\Delta + B_{\mathrm{added}}.$$

Stabilizing the unused-slot values requires f_{mult} to have a nonnegative fixed point:

$$\exists x \geq 0, \qquad f_{\mathrm{mult}}(x) = x.$$

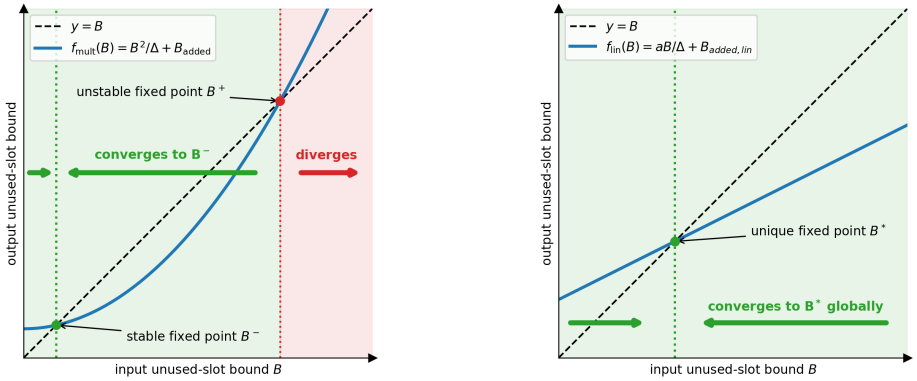
Since f_{mult} is convex, such a fixed point exists when $\Delta > 4B_{\mathrm{added}}$, for which the discriminant of $f_{\mathrm{mult}}(x) - x = 0$ is positive. Convergence is guaranteed when B is no more than the larger fixed point of f_{mult}.

Another basic circuit is a linear transformation of size at most L, i.e., at most L `GCKKS.HomRot` results are each multiplied by plaintext polynomials before being summed and rescaled. In this case, the old and new unused-slot bounds have a relationship in the form of

$$f_{\mathrm{lin}}(B) = aB/\Delta + B_{\mathrm{added,lin}}.$$

When $\Delta > a$, $f^i_{\mathrm{lin}}(x)$ converges globally to $B_{\mathrm{added,lin}}/(1 - a/\Delta)$.

The fixed point of f_{lin} should lie in the convergence region of f_{mult}, so that a circuit composed of these two kinds of operations has a stable unused-slot bound.



(a) Homomorphic multiplication.

(b) Linear transformation.

Fig. 2. The evolution of unused-slot bounds during homomorphic multiplication or a linear transformation.

These two sub-circuits encompass a variety of homomorphic operations. For example, a degree-$(L-1)$ polynomial evaluation consists of multiple homomorphic multiplications and one linear transformation, and `GCKKS.HomConj` can be regarded as a size-one linear transformation.

The full noise analysis. Let $c_m = \tau(F)[0]$ be the common useful-slot value of F, and let $c_e = \|\tau_{\text{unused}}(F)\|_\infty$ be its unused-slot bound.

We also define the variables useful for noise analysis. Let $V_{\text{enc}} = N\sigma_{\text{enc}}^2$ and $V_{\text{rnd}} = N/12$ be the per-slot variances of the RLWE encryption and rounding noises, respectively. For the RLWE secret s with Hamming weight h, the entries of $\tau(s)$ are modeled as $\mathcal{CN}(0,h)$. Its canonical norm is bounded by $\|s\|_{\text{can}} \le z\sqrt{h}$ with a failure probability of $\exp(-z^2)$. Let $V_{\text{ms}} = (1+\|s\|_{\text{can}}^2)V_{\text{rnd}}$ and V_{ks} be the per-slot variances of the modulus-switching and key-switching noises, respectively. Existing key-switching methods use RNS decomposition and an auxiliary modulus to control the size of key-switching noise [41], and the key-switching parameters can be set to easily ensure $V_{\text{ks}} < (1+\epsilon)V_{\text{ms}}$ for some small constant ϵ.

For a ciphertext $c = (b,a)$, define its unused-slot size after F-decryption as $U(c) = \left\|\tau_{\text{unused}}((b+as)F)\right\|_\infty$. The following lemma bounds the propagation of $U(c)$ through the two building blocks used in linear transformations and polynomial evaluations. Its proof is deferred to Appendix B.

Lemma 5 (Unused-slot noise bounds). *Suppose every input ciphertext satisfies $U(c) \le B$. The unused-slot sizes after a size-L linear transformation and after a `GCKKS.HomCMult` followed by rescaling are bounded respectively by*

$$f_{lin}(B) = A_L B + E_L \qquad \text{and} \qquad f_{mult}(B) = A_\times B^2 + E_\times,$$

where

$$A_L = \frac{z\sqrt{LV_{rnd}}\,c_e}{\Delta}, \qquad E_L = \frac{z^2 L\sqrt{V_{ks}V_{rnd}}\,c_e^2}{\Delta} + z\sqrt{V_{ms}}\,c_e,$$
$$A_\times = \frac{1}{\Delta}, \qquad E_\times = zc_e\sqrt{\frac{V_{ks}}{\Delta^2} + V_{ms}}.$$

Each complex Gaussian tail bound has a failure probability of $\exp(-z^2)$.

We now use the fixed points of f_{lin} and f_{mult} to give conditions for noise control over a GCKKS circuit. The map f_{lin} has a global nonnegative fixed point whenever $A_L < 1$. The quadratic map f_{mult} has two nonnegative fixed points whenever $4A_\times E_\times < 1$. Let Δ' denote the equivalent CKKS scale that is targeted for useful-slot noise growth.

Lemma 6 (Noise control conditions). *Denote the fixed point of f_{lin}, the larger fixed point of f_{mult}, and the unused-slot bound in a freshly encrypted ciphertext by*

$$B_{lin} = \frac{E_L}{1 - A_L}, \qquad B^+_{mult} = \frac{1 + \sqrt{1 - 4A_\times E_\times}}{2A_\times}, \qquad B_{fresh} = zc_e\sqrt{V_{enc} + V_{rnd}}.$$

Suppose that

$$A_L < 1, \qquad 4A_\times E_\times < 1, \qquad \max\{B_{lin}, B_{fresh}\} < B^+_{mult}, \qquad \frac{\Delta}{|c_m|} \geq \Delta'. \quad (7)$$

Then, every ciphertext c produced by a circuit consisting of size-L linear transformations and `GCKKS.HomCMult` *followed by rescaling satisfies $U(c) < B^+_{mult}$. Its useful slots meet the precision guaranteed by the corresponding CKKS noise analysis at scale Δ'.*

Theorem 2 (Minimum asymptotic scale). *Let $z, L, h \geq 1$, $k > 4$, $p \geq 0$, and $w = 2^{-p}$. The minimum scale satisfying the conditions of Lemma 6 has the asymptotic upper bound*

$$\Delta_{\min} = O\left(\delta_\beta^{(k-2)/2} \max\left\{ z\sqrt{kN}\,2^{4p/k} \max\left\{ \sqrt{zL}(h+1)^{1/4}, \max\{z\sqrt{h+1}, \sigma_{enc}\} \right\} \right.\right.$$
$$\left.\left. \Delta' 2^{-(1-4/k)p} \right\}\right). \quad (8)$$

Corollary 2. *Under the conditions of Theorem 2, define*

$$\mathcal{A} = \max\left\{ z^{3/2}\sqrt{L}(h+1)^{1/4}, z\max\{z\sqrt{h+1}, \sigma_{enc}\} \right\}.$$

Over $p \in \mathbb{R}_{\geq 0}$, the upper bound in (8) *is minimized by*

$$p^* = \max\left\{ 0, \log_2\left(\frac{\Delta'}{\sqrt{kN}\mathcal{A}} \right) \right\}.$$

If $\Delta' \geq \sqrt{kN}\mathcal{A}$*, its value at this choice is*

$$\Delta^*_{\min} = O\left(\delta_\beta^{(k-2)/2}(\sqrt{kN}\mathcal{A})^{1-4/k}(\Delta')^{4/k}\right). \tag{9}$$

Otherwise, $p^* = 0$ *and* $\Delta^*_{\min} = O(\delta_\beta^{(k-2)/2}\sqrt{kN}\mathcal{A})$.

Proof. The two p-dependent terms $\sqrt{kN}\mathcal{A}2^{4p/k}$ and $\Delta' 2^{-(1-4/k)p}$ change in opposite directions with p. Balancing them gives $2^p = \Delta'/(\sqrt{kN}\mathcal{A})$. When the ratio is below one, the encoding starts to trade encoding precision for smaller unused-slot size. In this case, p is set to zero to prevent loss in encoding precision. Substitution gives the claimed bounds.

4.3 Using an Invertible F

As an optimization, we show that restricting F to R-units removes the need for an extra multiplicative layer in `GCKKS.HomConj`.

Depth-free homomorphic conjugation. Recall that `GCKKS.HomConj` uses one `GCKKS.HomPMult` to cancel the extra $c_m\bar{c}_m^{-1}$ factor caused by conjugation. When F is a unit, this `GCKKS.HomPMult` can be replaced with a polynomial multiplication by $\texttt{Conj}(F) \cdot F^{-1}$. This operation neither amplifies the noise nor consumes any depth because $\|\texttt{Conj}(F) \cdot F^{-1}\|_{\text{can}} = 1$.

A subtle issue is that the operation `GCKKS.HomConj` cannot be captured by a homomorphic linear transformation, making it incompatible with the noise analysis model. However, this can be mostly addressed by adopting lazy rescaling, so that conjugation is merged into the preceding linear-transformation or ciphertext-squaring sub-circuit. Although the key-switching in `GCKKS.HomConj` adds extra noise, the noise becomes negligible after the subsequent rescaling, thus having little impact on the validity of the noise model.

Finding a unit F in the log-unit lattice. The F constructed from $\mathbf{b}_0^*$ may not be invertible, so we exploit the log-unit lattice to find a unit $F \in \mathbb{Z}[Y]/(Y^{k/2}+1)$.

Let $n = k/2$ and $R = \mathbb{Z}[Y]/(Y^n+1)$. The logarithm embedding is defined by

$$\text{Log} : R \to \mathbb{R}^{n/2}, \qquad \text{Log}(m) = (\ln|\tau(m)[i]|)_{i\in[n/2]}.$$

Dirichlet's unit theorem [51] implies that $\text{Log}(R^*)$ is a lattice, often called the log-unit lattice. For a power-of-two n, Weber's class number conjecture [57] states that $\text{Log}(R^*)$ is generated by the cyclotomic units of

$$X, \text{ and } u_i = \frac{X^{5^i}-1}{X-1}, \qquad 1 \leq i \leq n/2-1.$$

$\text{Log}(R^*)$ has rank $n/2-1$ and is orthogonal to the all-one vector $\mathbf{1}$.

The search for F with p-bit encoding precision reduces to solving approximate CVP in $\mathrm{Log}(R^*)$ with the target

$$p \ln 2 \cdot \left(-(1-4/k),\ \ 4/k,\ \ \ldots,\ \ 4/k\right)^T \in \mathrm{span}(\mathrm{Log}(R^*)).$$

Suppose the CVP solution has a coefficient vector of $\mathbf{c} \in \mathbb{Z}^{n/2-1}$, then the corresponding unit F is recovered as $F = \prod_{i \in [n/2-1]} u_i^{\mathbf{c}[i]}$.

Cramer et al. showed that applying stochastic rounding to the basis coefficients gives good CVP solutions in $\mathrm{Log}(R^*)$ [28].

Theorem 3 (Theorem 6.3 from [28]). *Assuming Weber's conjecture, there is an efficient randomized algorithm that, given any vector $\mathbf{x} \in span(Log(R^*))$, outputs a vector $\mathbf{v} \in Log(R^*)$ such that $\|\mathbf{x} - \mathbf{v}\|_\infty = O(\sqrt{n \log n})$.*

For fixed k, both approaches achieve the same asymptotic dependence on w with different constant factors. Applying BKZ reduction and Babai rounding to the log-unit lattice yields a worse asymptotic bound in k, under GSA and for fixed block size, than the $\mathbf{B}_{k,w}$ approach[7]. Empirical data on the quality of unit F are given in Appendix E, which shows that it becomes harder to find unit F with encoding quality as good as those from $\mathbf{B}_{k,w}$ when k is large.

5 Experiments

We provide a proof-of-concept implementation of GCKKS using fpylll [56] and Lattigo [48]. All Lattigo experiments are conducted in a single thread.

5.1 Implementation Details on Lattice Reduction.

We use fpylll to perform LLL/BKZ reduction and Babai rounding. The real-valued lattice $\mathbf{B}_{k,w}$ is first multiplied by a large factor S and then rounded into an integer lattice before lattice reduction. Practical values of w are extremely small and thus vulnerable to rounding errors introduced by lattice discretization. Therefore, we choose $w = 2^{-p}$ for some $p \in \mathbb{Z}$ and use only S satisfying $wS \in \mathbb{Z}$ to eliminate any rounding error on w.

We use $S = 2^{1200}$ in the case of $k = 256$ and $-\log_2(w) \approx 1000$. The maximum BKZ block size for reducing $\mathbf{B}_{k,w}$ is set to 40.

5.2 Experiment Results

Following the settings from both BLEACH and GBFV, we use a ring dimension of $N = 2^{14}$, a ternary RLWE secret of Hamming weight $h = 256$, and an RLWE

[7] The first version of this paper incorrectly claimed that the log-unit lattice search always gives asymptotically worse results. A more accurate statement is that this worse encoding quality is present when BKZ/GSA is also applied to the log-unit lattice search. For stochastic-rounding-based search, its asymptotic bound is better than the search in $\mathbf{B}_{k,w}$.

modulus around 420 bits[8]. We set the tail bound parameter to $z = 10$, corresponding to a per-event failure probability of 2^{-144}. The encryption error has a standard deviation of $\sigma_{\text{enc}} = 3.2$. The unit-$F$ optimization is not adopted.

Arithmetic in ultra-high precision. We show how to use GCKKS to realize homomorphic computation with extremely high precision[9]. The selected parameters are summarized in Table 1. Here, $\Delta' = \Delta/|c_m|$ denotes the equivalent CKKS scale, and L stands for the maximum linear-transformation size. ϵ defines the key-switching noise size relative to rescaling noise, namely $V_{\text{ks}} \leq (1+\epsilon)V_{\text{ms}}$. The modulus chain provides seven multiplication levels.

Table 1. Parameters for the Lattigo experiment.

N	k	Slots	$\log_2 w^{-1}$	$\log_2 \Delta'$	$\log_2(\Delta)$	$\log_2 Q_i$	$\log_2 P_i$	d_{num}	$\log_2 QP$	L	ϵ
2^{14}	256	128	1004	1050	37	$(40, 37 \times 7)$	$(60, 60)$	3	419	32	0.44

The initial square-and-rescale preserves more than 1000 bits of precision and takes 28.16 ms, averaged over 1000 iterations. To our knowledge, this is the first time for approximate homomorphic encryption to reach such precision.

Reducing modulus consumption. Using the same N, L, z, and ϵ from above, we calculate the minimum Δ corresponding to $\log_2 \Delta' \in \{20, \ldots, 100\}$ and $k \in \{8, 16, 32, 64\}$. The results are plotted in Figure 3.

The slopes of the curves agree well with the predicted values $4/k$, and interestingly, all four curves seem to intersect $y = x$ at the same point. This is consistent with the prediction of Theorem 2: taking logarithms of both sides of (9) and omitting the δ_β term, the $\log_2 \Delta - \log_2 \Delta'$ curve intersects $y = x$ at $\log_2(\sqrt{kN}\mathcal{A})$.

5.3 Remark on Efficiency Issues

Preprocessing time. The lattice reduction is efficient for small and moderate choices of k. For $k \leq 64$, BKZ reduction finishes almost instantly given the small

[8] This parameter set is insecure under the recent attack on sparse-secret RLWE [42]. Based on this attack, the estimator by Dumezy gives $\log_2(QP) \leq 387$ under the choices of N and h [32]. One 37-bit layer can be removed from the RNS chain to make the parameter set secure again.

[9] This parameter set is only intended to demonstrate the capability of GCKKS to support high-precision arithmetic with reduced modulus consumption. It does not include the extra exponentially large modulus budget required by the noise flooding in circuit privacy. The noise flooding in IND-CPA^D only acts on the decrypted useful slots, so it does not require an additional modulus budget and is compatible with this parameter set.

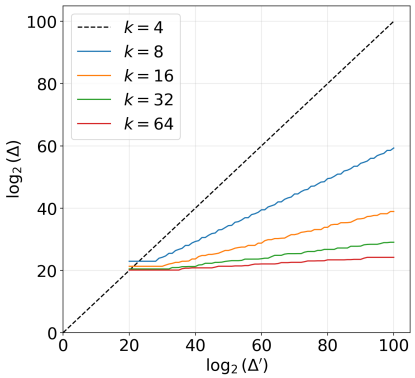


Fig. 3. $\log_2 \Delta$ – $\log_2 \Delta'$ graph for different k.

lattice dimension; for $k = 256$, BKZ finishes in less than an hour. Moreover, as the lattice reduction needs to be computed only once and is then cached for later use, a longer preprocessing time is usually acceptable.

Use of high-precision data types. The implementation already moves all high-precision lattice reduction into the offline phase. However, high-precision floating-point arithmetic is still required for encoding and decoding and for tracking the evolution of the scaling factor during homomorphic operations. We believe this requirement is inherent in pursuing high-precision homomorphic computation and expect implementation-level optimizations to reduce this overhead.

A longer RNS chain. Due to the reduced modulus consumption of GCKKS, $\log_2(\Delta)$, the bit length of the scaling factor, can be far smaller than the machine word size, as shown in Figure 3. This increases the number of RNS limbs to represent ciphertexts, leading to extra computational overhead. Therefore, real-world applications should adopt techniques like Grafting [22] to reduce the stored RNS limbs for better performance.

6 Conclusion and Discussion

For deep circuits with high-precision approximate arithmetic, GCKKS offers significant improvements over existing homomorphic encryption schemes. However, it is not yet fully homomorphic because it lacks an efficient bootstrapping algorithm. We view this as an important but challanging task, and plan to address it in our subsequent work.

Acknowledgement

We thank Jules Dumezy for his helpful comments and discussion on the comparison of the two works, and on the security issues with our parameter setting and its compatibility with noise flooding.

A Generative AI Usage

We used ChatGPT 5.4, 5.5, and 5.6 in this work. We detail the contributions of AI and the authors below.

Constructions. All constructions, including the lattice- and polynomial-based encoders, the F-decryption format, the use of an invertible F, the design of homomorphic operations, and the noise-control model, were developed by the authors.

AI improved the search for a unit F by suggesting the use of the log-unit lattice, whereas our initial algorithm enumerated $\mathcal{L}(\mathbf{B}_{k,w})$ to look for units. Using the log-unit lattice halves the lattice rank and guarantees that all solutions are units, so it is much faster than our initial design. The log-unit approach also allowed us to reuse results from [28] to analyze the existence and quality of a unit F and the cost of finding one, whereas our previous analysis relied on an additional assumption.

Proof. The initial proofs of Proposition 1 and Lemma 4 were provided by the authors. They assumed coordinate-wise independence after projection and a spherical-Gaussian model for the Babai error, and were heuristic.

AI suggested using Ky Fan's principle in the proof of Proposition 1, thereby avoiding the heuristic assumptions in our initial proof. AI also drafted the proofs of simpler results, such as Lemma 5 and Lemma 6.

All proofs have been manually checked by the authors.

Coding. The authors wrote the first working version of the fpylll-based plaintext encoder and a procedure for selecting the GCKKS scaling factor.

Based on this version, AI helped improve implementation efficiency and robustness, migrate the implementation to HEAAN and Lattigo, and write scripts for proposition validation and benchmarking.

Writing. Most of the text was written by the authors, and AI was used to summarize experimental data and proofread the manuscript. All AI-generated text has been manually checked.

B Proofs

Proof (Lemma 5). Consider a fixed unused slot. In a size-L linear transformation, let t_i be the unused-slot error in the i-th rotated ciphertext and let r_i be the unused-slot value of the corresponding encoded plaintext coefficient. The input bound gives $|t_i| \le B$, while the rounding-noise model gives

$$\mathrm{Var}(r_i) \le V_{\mathrm{rnd}} c_e^2, \qquad \mathrm{Var}(e_{\mathrm{ks},i}) \le V_{\mathrm{ks}} c_e^2.$$

After the inner product and rescaling, the unused slot contains

$$\frac{1}{\Delta}\sum_{i\in[L]}(t_i+e_{\mathrm{ks},i})r_i+e_{\mathrm{ms}}=\frac{1}{\Delta}\sum_{i\in[L]}t_ir_i+\frac{1}{\Delta}\sum_{i\in[L]}e_{\mathrm{ks},i}r_i+e_{\mathrm{ms}}.$$

The first summation has a standard deviation at most $B\sqrt{LV_{\mathrm{rnd}}}c_e/\Delta$, which gives the term A_LB. After bounding $e_{\mathrm{ks},i}$, r_i and e_{ms} by z times their standard deviation, the second and third terms are bounded respectively by

$$\left|\frac{1}{\Delta}\sum_{i\in[L]}e_{\mathrm{ks},i}r_i\right|\le\frac{z^2L\sqrt{V_{\mathrm{ks}}V_{\mathrm{rnd}}}\,c_e^2}{\Delta},\qquad|e_{\mathrm{ms}}|\le z\sqrt{V_{\mathrm{ms}}}c_e,$$

giving the two terms of E_L.

For ciphertext multiplication, after rescaling, the output error at an unused slot is

$$\frac{t_0t_1}{\Delta}+\frac{e_{\mathrm{ks}}F}{\Delta}+e_{\mathrm{ms}}F.$$

The first term is bounded by $B^2/\Delta=A_\times B^2$. The remaining two centered Gaussian terms have combined variance at most

$$c_e^2\left(\frac{V_{\mathrm{ks}}}{\Delta^2}+V_{\mathrm{ms}}\right),$$

which gives $E_\times$. □

Proof (Lemma 6). The first two conditions in (7) ensure that the fixed points in the statement exist and are nonnegative. Moreover, for every $0\le B<B^+_{\mathrm{mult}}$, monotonicity and $f_{\mathrm{mult}}(B^+_{\mathrm{mult}})=B^+_{\mathrm{mult}}$ give $f_{\mathrm{mult}}(B)<B^+_{\mathrm{mult}}$. For the linear map, write

$$f_{\mathrm{lin}}(B)=A_LB+(1-A_L)B_{\mathrm{lin}}.$$

Because $0\le A_L<1$ and both B and B_{lin} are smaller than B^+_{mult}, we also have $f_{\mathrm{lin}}(B)<B^+_{\mathrm{mult}}$. Thus $[0,B^+_{\mathrm{mult}})$ is invariant under both propagation maps. A fresh ciphertext lies in this interval because $U(c)\le B_{\mathrm{fresh}}<B^+_{\mathrm{mult}}$. Induction over a topological ordering of the circuit proves the unused-slot claim.

The noise statement for useful slots directly follows from Theorem 1. □

Proof (Theorem 2). Set

$$a=z\sqrt{LV_{\mathrm{rnd}}}\,c_e,\qquad d=z^2L\sqrt{V_{\mathrm{ks}}V_{\mathrm{rnd}}}\,c_e^2,$$
$$e=z\sqrt{V_{\mathrm{ms}}}\,c_e,\qquad q=zc_e\sqrt{V_{\mathrm{ks}}}.$$

By Lemma 5,

$$A_L=\frac{a}{\Delta},\qquad E_L=\frac{d}{\Delta}+e,\qquad A_\times=\frac{1}{\Delta},\qquad E_\times\le\frac{q}{\Delta}+e.$$

Define

$$D=\max\left\{a,\sqrt{d},e,\sqrt{q},zc_e\sqrt{V_{\mathrm{enc}}+V_{\mathrm{rnd}}},|c_m|\Delta'\right\}.$$

Choose Δ to be a sufficiently large constant multiple of D. Since it dominates a, we have $A_L < 1$ bounded away from one. Since it also dominates $\sqrt{q}$ and e, we obtain $4A_\times E_\times < 1$.

For a sufficiently large hidden constant, $1 - A_L \geq 1/2$, and hence

$$B_{\text{lin}} = \frac{E_L}{1 - A_L} \leq 2\left(\frac{d}{\Delta} + e\right) < \frac{\Delta}{2}.$$

The last inequality follows because Δ dominates both $\sqrt{d}$ and e. On the other hand, the larger multiplication fixed point satisfies $B^+_{\text{mult}} \geq \Delta/2$, so $B_{\text{lin}} < B^+_{\text{mult}}$. Similarly, Δ dominates $B_{\text{fresh}} = zc_e\sqrt{V_{\text{enc}} + V_{\text{rnd}}}$, which gives $B_{\text{fresh}} < B^+_{\text{mult}}$. Finally, it dominates $|c_m|\Delta'$, so $\Delta/|c_m| \geq \Delta'$. Thus $\Delta_{\min} = O(D)$.

Finally, since $p \geq 0$ and $k > 4$,

$$w^2 + \frac{k}{4} = \Theta(k), \qquad \left(\frac{k}{4}\right)^{4/k} = \Theta(1).$$

Therefore, Lemma 4 gives

$$c_e = O\left(\sqrt{k}\,\delta_\beta^{(k-2)/2} 2^{4p/k}\right), \qquad |c_m| = O\left(\delta_\beta^{(k-2)/2} 2^{-(1-4/k)p}\right).$$

Substituting $V_{\text{rnd}} = \Theta(N)$, $V_{\text{ks}}, V_{\text{ms}} = O(z^2N(h+1))$, and $V_{\text{enc}} = N\sigma^2_{\text{enc}}$ into D yields (8). Since $z, h \geq 1$, a is dominated by $\sqrt{d}$. The $\sqrt{q}$ term is also dominated by the third term in the inner maximum. Hence neither term appears in (8).

□

Remark. In fact, we can prove the stronger result $\Delta_{\min} = \Theta(D)$. However, this does not translate into an asymptotically tight bound on Δ because Lemma 4 may not be tight up to constant factors.

C Empirical Verification of Propositions

A BKZ block size of $\beta = \min(k/2, 40)$ is used across all experiments.

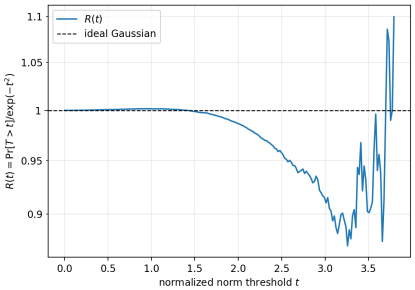
For Proposition 1, we ran the encoding algorithm 1,000,000 times with parameters $N = 256$, $k = 16$, $w = 2^{-40}$, $\beta = 8$, and $\Delta \approx \sqrt{V_{\text{unused}}}$. This produced 3.2×10^7 useful-slot encoding errors and 9.6×10^7 unused-slot values. For a slot variable Z, let $\mu = \mathrm{E}[Z]$ and $\sigma_Z^2 = \mathrm{E}\,|Z - \mu|^2$. We measure its normalized mean μ/σ_Z, circularity coefficient [33,53]

$$\rho_Z = \frac{\mathrm{E}[(Z-\mu)^2]}{\mathrm{E}\,|Z-\mu|^2},$$

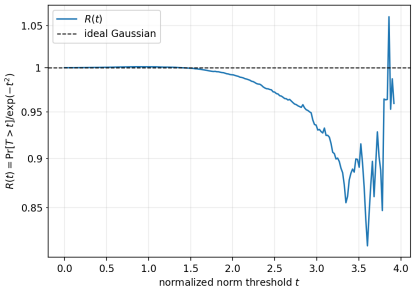
and standard-deviation ratio $\sigma_Z/\sqrt{V^*}$. Here V^* is the variance predicted by directly evaluating the GSO covariance, without the Ky Fan or canonical-embedding bounds used in Proposition 1. For unused slots, both the measured and predicted variances in this ratio are maximized over the unused slots.

Table 2. Distribution statistics for lattice-based encoding.

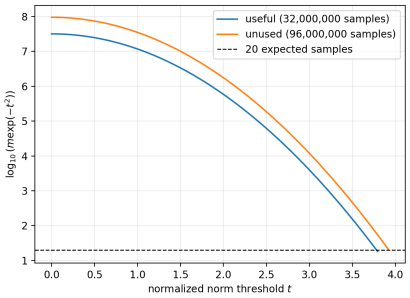
Slot variable	$\mathrm{Re}(\mu/\sigma_Z)$	$\mathrm{Im}(\mu/\sigma_Z)$	$\lvert\rho_Z\rvert$	$\sigma_Z/\sqrt{V^*}$
Useful-slot error	-1.563×10^{-4}	-8.573×10^{-5}	5.239×10^{-3}	1.00001
Unused-slot value	-7.784×10^{-5}	1.590×10^{-4}	9.775×10^{-4}	1.00129
Ideal case	0	0	0	1



(a) Useful-slot errors.



(b) Unused-slot values.



(c) Expected tail-sample counts under the ideal model.

Fig. 4. Empirical Gaussian-tail validation. Figures (a) and (b) plot the tail-probability ratio $R(t)$; figure (c) shows the corresponding ideal-Gaussian expected sample counts.

These metrics are summarized in Table 2 and support modeling Z as a centered, circularly symmetric variable with variance close to the prediction.

To assess whether the distribution is close to Gaussian, we compare the normalized tail probabilities of Z with those of a standard complex Gaussian distribution through

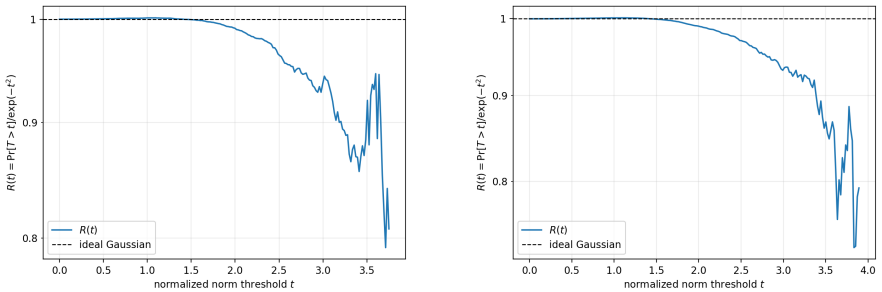
$$R(t) = \frac{\Pr[|Z-\mu|/\sigma_Z > t]}{\exp(-t^2)}.$$

The first two subfigures of Figure 4 show that $R(t)$ remains close to one over the well-sampled range. Its fluctuations increase rapidly near the end of each curve, because the number of tail samples has become too small in this region.

For Assumption 3, we used the same parameters and number of encodings to test deterministic nearest-integer coefficient rounding. The collected statistics are reported below. For unused slots, the last column in the table reports the largest per-slot standard deviation.

Table 3. Distribution statistics for the entries of $\tau(M - M')$.

Entry set	$\mathrm{Re}(\mu/\sigma_Z)$	$\mathrm{Im}(\mu/\sigma_Z)$	$\lvert\rho_Z\rvert$	$\sigma_Z/\sqrt{N/12}$
Useful slots	-1.027×10^{-4}	-5.237×10^{-5}	3.614×10^{-4}	0.999997
Unused slots	1.758×10^{-5}	6.772×10^{-6}	1.225×10^{-4}	1.00140
Ideal case	0	0	0	1



(a) Entries at useful slots. (b) Entries at unused slots.

Fig. 5. Empirical Gaussian-tail validation for $\tau(M-M')$ under deterministic coefficient rounding. Both panels plot the tail-probability ratio $R(t)$ defined above.

The normalized means and circularity coefficients in Table 3 are close to zero, and the standard deviations agree with $\sqrt{N/12}$. In Figure 5, both tail ratios remain close to one until falling mildly below one in the far tail. The last few points fluctuate more strongly because few samples reach these thresholds. These results support the centered, circularly symmetric Gaussian approximation used in Corollary 1.

Finally, we note that the bounds in Proposition 1 and Lemma 4 depend only on GSO norms and discard directional information. When the complete GSO is available, its projection and canonical embeddings can instead be evaluated directly to obtain tighter, basis-specific estimates. We empirically measured the looseness introduced by the worst-case GSO-orientation and canonical-embedding bounds. The results are given in Appendix D.

D Looseness of the Encoding-Noise Bounds

This appendix quantifies the looseness introduced by the Ky Fan and canonical-embedding bounds in Proposition 1 and Lemma 4. These bounds are required because the orientation and canonical-embedding values of GSO vectors are unknown a priori. However, once the GSO vectors are fixed, these bounds can be replaced by accurate calculations, yielding tighter predictions of encoding errors.

D.1 Lattice-based encoding

For Proposition 1, we compare its useful- and unused-slot variance bounds with the predicted variances obtained from known GSO vectors. We also separate the contributions of the Ky Fan and canonical-embedding bounds. The results are summarized in Figure 6.

The looseness of both the useful- and unused-slot variance bounds increases with k. The Ky Fan bound has a larger effect on useful slots than on unused slots, whereas the canonical-embedding bound is the main source of looseness for unused slots.

D.2 Polynomial-multiplication-based encoding

For Lemma 4, define the exact useful- and unused-slot reference values as

$$B^*_{\text{use}} = |c_m|^2, \qquad B^*_{\text{unused}} = \max_{\xi \text{ unused}} |F(\xi)|^2.$$

For useful slots, we compare $\alpha\|\mathbf{b}_0^*\|^2$ with B^*_{use}. For unused slots, the combined bound is $n\|\mathbf{b}_0^*\|^2/w^2$. To separate its two sources of looseness, we also evaluate the exact-projection plus canonical-embedding bound

$$\frac{n}{w^2}\left(\|\mathbf{b}_0^*\|^2 - |c_m|^2\right)$$

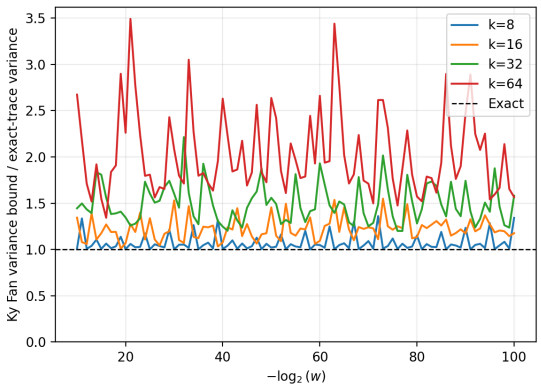
and the Ky Fan plus exact-embedding estimate obtained by scaling B^*_{unused} by

$$\frac{\|\mathbf{b}_0^*\|^2}{\|\mathbf{b}_0^*\|^2 - |c_m|^2}.$$

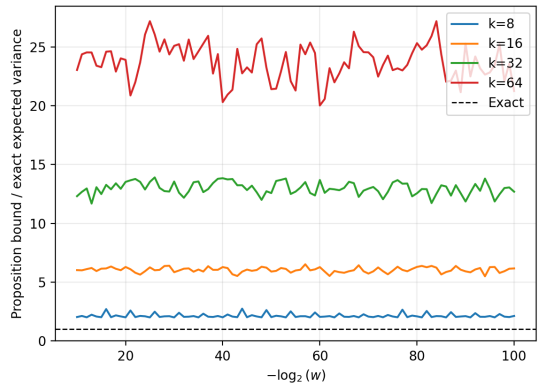
All reported ratios use B^*_{use} or B^*_{unused} as the denominator. The same ratios apply to the variances in Corollary 1, because their common factor $N/12$ cancels.

Figure 7 shows that discarding the orientation of $\mathbf{b}_0^*$ can make the useful-slot bound substantially loose, especially for larger k. The combined unused-slot bound is more stable but remains several times larger than the exact reference. The decomposition for $k = 64$, representative of the other tested dimensions, shows that the canonical-embedding bound is the dominant source of looseness for unused slots, whereas the Ky Fan contribution remains close to one.

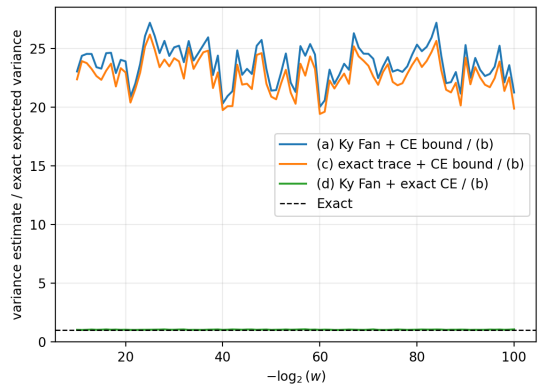
The polynomial-multiplication-based bound is looser than the lattice-based bound because it discards the orientation of only one GSO vector rather than averaging the projections of all GSO vectors. Consequently, the useful-slot bound can be particularly conservative when $\mathbf{b}_0^*$ is nearly orthogonal to V.



(a) Looseness of the Ky Fan bound for useful-slot variance.

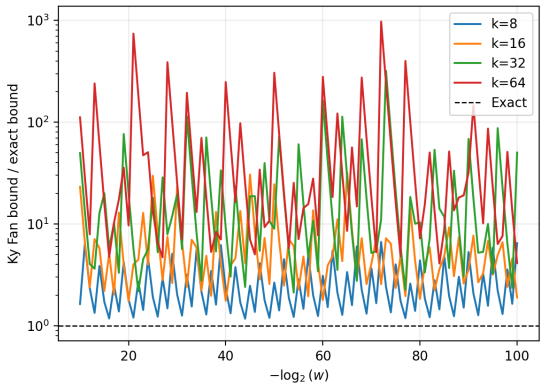


(b) Combined looseness for unused-slot variance.

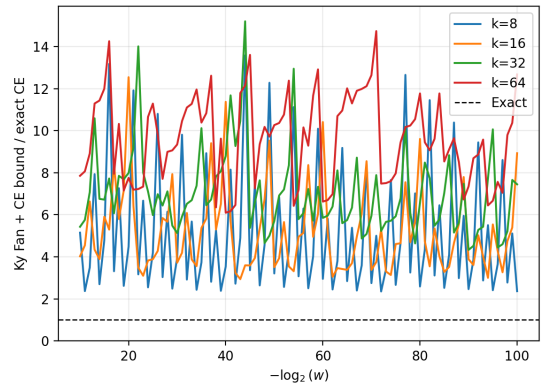


(c) Separate looseness of the two bounds for $k = 64$.

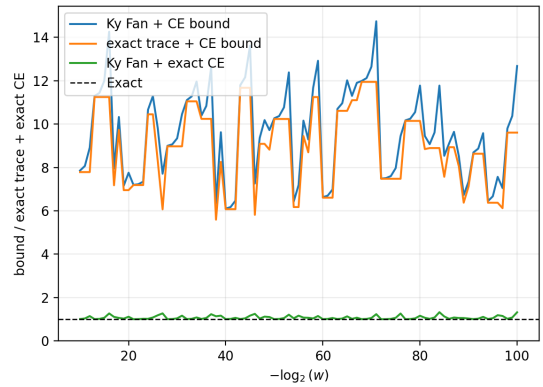
Fig. 6. Looseness introduced by the Ky Fan and canonical-embedding bounds in Proposition 1, measured as the ratio of each bound to the corresponding exact reference value.



(a) Ky Fan looseness for useful-slot values.



(b) Combined looseness for unused-slot values.



(c) Separate looseness of the two bounds for $k = 64$.

Fig. 7. Bound looseness in Lemma 4, measured as the ratio of each bound to the corresponding exact reference value. The useful-slot plot uses a logarithmic vertical axis.

E Quality of Unit F

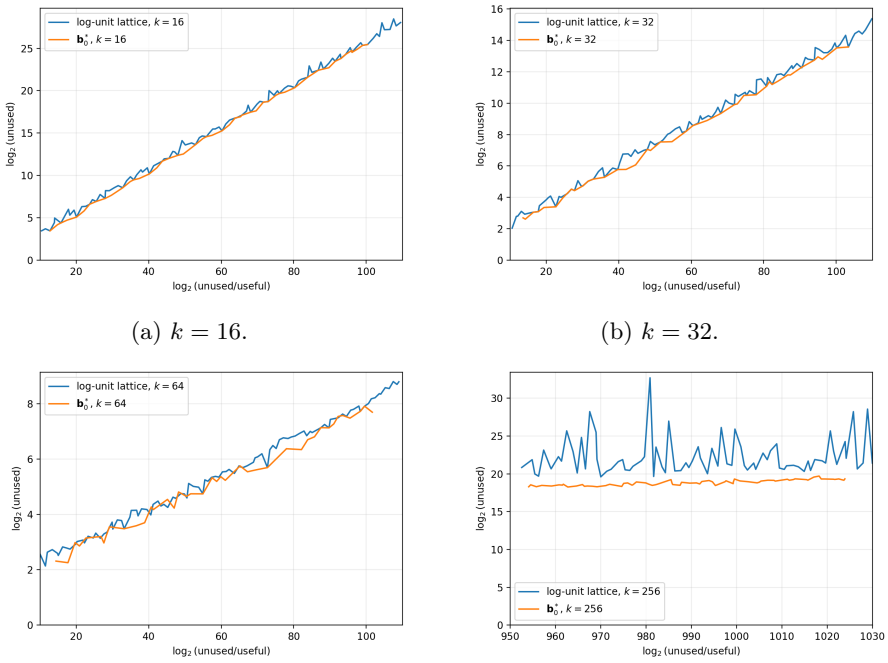
For $k \in \{8, 16, 32, 64\}$ with $\log_2(w^{-1}) \in \{10, \ldots, 100\}$, and $k = 256$ with $\log_2(w^{-1}) \in \{950, \ldots, 1050\}$, we enumerate the log-unit lattice around the CVP target to search for F. We set a running time limit of 3 hours on the enumeration program for $k = 256$, while the limits for the other values of k are set to 10 minutes. The upper limit of lattice points returned by enumeration is set to 10^6.

After collecting these unit choices of F, we assess their quality by comparing the resulting $\log_2(c_e) - \log_2(c_e/|c_m|)$ curve with the corresponding curve for non-unit choices of F constructed from $\mathbf{b}_0^*$. Multiple F polynomials may have close values of $\log_2(c_e/|c_m|)$, but only the one with the smallest $\log_2(c_e)$ is displayed.

In the very special case of $k = 8$, $\mathrm{Log}(R^*)$ is a one-dimensional lattice generated by (the logarithm of) $u_1 = (X^5 - 1)/(X - 1)$. It can be shown that

$$|\tau(u_1)[0]| = |\tau(u_1)[1]|^{-1} = 1 + \sqrt{2} \approx 2^{1.27},$$

so the powers of u_1 can be used to construct unit choices of F with a bit granularity of 1.27. Therefore, the $\log_2(c_e) - \log_2(c_e/|c_m|)$ curve has the exact shape of $y = x/2$.



(a) $k = 16$. (b) $k = 32$.

(c) $k = 64$. (d) $k = 256$.

Fig. 8. $\log_2(c_e) - \log_2(c_e/|c_m|)$ curves for unit and non-unit choices of F.

For $k > 8$, unit choices of F generally have worse quality than non-unit choices, as expected. Namely, a unit F requires a larger $\log_2(c_e)$ to realize the same $\log_2(c_e/|c_m|)$ as a non-unit F. This gap becomes larger as k increases, because the increased log-unit lattice volume degrades the quality of F, and the increased cost of lattice enumeration also makes it difficult to find a large number of F candidates. The $\log_2(c_e)$ – $\log_2(c_e/|c_m|)$ curves are presented in Figure 8.

F Our Early Attempts

F.1 An Alternative Perspective on the Optimal CVP Target

Recall that in the coefficient-wise rounding of $m' \in \mathbb{C}[X]/(X^{2N/k} - \zeta_k)$, the original choice of CVP target

$$\big(\mathrm{Re}(m'[i])\ \ \mathrm{Im}(m'[i])\ \ 0\ \ \cdots\ \ 0\big)^T$$

has a nonzero projection onto $\mathrm{span}(\mathbf{B}_{k,w})^\perp$, causing extra rounding error.

Our first attempt to solve this issue was to replace the target with some $\mathbf{t}'_i \in \mathrm{span}(\mathbf{B}_{k,w})$. We restrict our choices to $\mathbf{t}'_i$ whose first two coordinates remain $\mathrm{Re}(m'[i])$ and $\mathrm{Im}(m'[i])$, in order to preserve useful-slot precision. Then, we minimize the ℓ_2-norm of $\mathbf{t}'_i[2:]$ to minimize the unused-slot magnitude.

Suppose $\mathbf{t}'_i = \mathbf{B}_{k,w}\mathbf{x}'_i$. By the expression of $\mathbf{B}_{k,w}$, $\mathbf{t}'_i[2:] = w\mathbf{x}'_i$. Therefore, minimizing $\|\mathbf{t}'_i[2:]\|_2$ is equivalent to finding the least-norm solution $\mathbf{x}'_i$ of

$$\begin{pmatrix}\mathbf{h}_0^T\\ \mathbf{h}_1^T\end{pmatrix}\mathbf{x} = \begin{pmatrix}\mathrm{Re}(m'[i])\\ \mathrm{Im}(m'[i])\end{pmatrix},$$

which is given by the pseudoinverse

$$\begin{aligned}\mathbf{x}'_i &= \begin{pmatrix}\mathbf{h}_0 & \mathbf{h}_1\end{pmatrix}\left(\begin{pmatrix}\mathbf{h}_0^T\\ \mathbf{h}_1^T\end{pmatrix}\begin{pmatrix}\mathbf{h}_0 & \mathbf{h}_1\end{pmatrix}\right)^{-1}\begin{pmatrix}\mathrm{Re}(m'[i])\\ \mathrm{Im}(m'[i])\end{pmatrix}\\ &= \frac{4}{k}\big(\mathrm{Re}(m'[i])\mathbf{h}_0 + \mathrm{Im}(m'[i])\mathbf{h}_1\big)\\ &= \frac{4}{k}\big(\mathrm{Re}(m'[i]\zeta_k^{-j})\big)^T_{j\in[k/2]}.\end{aligned}$$

Multiplying $\mathbf{x}'_i$ by w gives $\mathbf{t}'_i[2:]$.

Now, we verify that $\mathbf{t}'_i$ agrees with the optimal CVP target obtained from $\mathtt{Lift}_{N,k}(m')$. Note that $\mathbf{t}'_i[0] = \mathrm{Re}(m'[i])$ and $\mathbf{t}'_i[1] = \mathrm{Im}(m'[i])$ already agree with the optimal CVP target $\mathbf{t}_i$, so we only need to check $\mathbf{t}'_i[2:]$. Let

$$Q(X) = \frac{X^{N/2} - J}{X^{2N/k} - \zeta_k} = \sum_{j=0}^{k/4-1} \zeta_k^{k/4-1-j} X^{j\cdot 2N/k}.$$

It satisfies $Q(X) \bmod (X^{2N/k} - \zeta_k) = (k/4)\zeta_k^{k/4-1}$. Thus, before identifying J with $X^{N/2}$, the CRT lift sends m' to

$$m'Q\big(Q \bmod (X^{2N/k} - \zeta_k)\big)^{-1} = \frac{4}{k}m' \sum_{j=0}^{k/4-1} \zeta_k^{-j} X^{j\cdot 2N/k}.$$

Then, replacing J with $X^{N/2}$ gives

$$M' = \frac{4}{k}\sum_{j=0}^{k/4-1}\sum_{i=0}^{2N/k-1}\mathrm{Re}(m'[i]\zeta_k^{-j})X^{i+j\cdot 2N/k} + \frac{4}{k}\sum_{j=0}^{k/4-1}\sum_{i=0}^{2N/k-1}\mathrm{Im}(m'[i]\zeta_k^{-j})X^{i+j\cdot 2N/k}\cdot X^{N/2}$$
$$= \frac{4}{k}\sum_{j=0}^{k/2-1}\sum_{i=0}^{2N/k-1}\mathrm{Re}(m'[i]\zeta_k^{-j})X^{i+j\cdot 2N/k}.$$

Finally, recall $\mathbf{t}_i[2:] = w\cdot(M'[i+j\cdot 2N/k])_{j\in[k/2]}$. Substituting the coefficients of M' gives $\mathbf{t}_i = \mathbf{t}'_i$.

This observation also suggests that M' has correlated coefficients, which makes it necessary to validate Assumption 3.

F.2 Homomorphic Operations without F Polynomials

Recall that homomorphic operations such as modulus switching and encryption introduce noise with standard deviation $\Omega(\sqrt{N})$ into slots.

Our first attempt to mitigate the issue was to re-encode the modulus switching error using the Babai-based encoder, then subtract the result from the mod-switched ciphertext.

The modulus switching noise has the form of $e_{\mathrm{ms}} = b_{\mathrm{ms}} + a_{\mathrm{ms}}s$, with s the RLWE secret and $b_{\mathrm{ms}}, a_{\mathrm{ms}} \in \mathbb{R}[X]/(X^N+1)$ and $\|b_{\mathrm{ms}}\|_\infty, \|a_{\mathrm{ms}}\|_\infty \le 0.5$. We first reduce a_{ms} and b_{ms} modulo $X^{2N/k}-\zeta_k$ to obtain $\bar{a}_{\mathrm{ms}}$ and $\bar{b}_{\mathrm{ms}}$. Notice that this step preserves the useful slot values of e_{ms}, because

$$\tau_{\mathrm{useful}}(\bar{b}_{\mathrm{ms}} + \bar{a}_{\mathrm{ms}}s) = \tau_{\mathrm{useful}}(b_{\mathrm{ms}} + a_{\mathrm{ms}}s).$$

Then, we round both $\bar{a}_{\mathrm{ms}}$ and $\bar{b}_{\mathrm{ms}}$ from $\mathbb{C}[X]/(X^{2N/k}-\zeta_k)$ into $\mathbb{Z}[\zeta_k][X]/(X^{2N/k}-\zeta_k) \cong \mathbb{Z}[X]/(X^N+1)$, in the same way as the encoding process does. The rounded values, denoted by a'_{ms} and b'_{ms}, are then subtracted from the RLWE mask in the mod-switched ciphertext. This operation approximately subtracts $\tau_{\mathrm{useful}}(e_{\mathrm{ms}})$ from the slots, thereby preventing e_{ms} from ruining the useful-slot precision. We remark that the re-encoding error in a'_{ms} is amplified by its multiplication by s, so the modulus-switching noise is larger than fresh encoding noise.

For encryption noise (including that in key-switching keys), we could increase the scaling factor by some factor P during plaintext encoding, encrypt the plaintext polynomial, and mod-switch the ciphertext by P. The modulus-switching error must be re-encoded as discussed above. With $P = \Omega(w^{-1+4/k})$, the encryption noise in the useful slots after modulus switching by P will have a magnitude comparable to the modulus-switching noise with re-encoding. Therefore, the encryption introduces small enough errors on useful slots as desired.

This approach only makes homomorphic computation *feasible*, but not *efficient*. It has drawbacks similar to Tuple-CKKS, namely significant computational overhead and modulus waste. Concerning computational cost, each modulus-switching operation now requires $4N/k$ Babai-rounding operations in

dimension $k/2$, corresponding to a computational cost of $O(Nk)$. This overhead becomes heavier as the encoding precision increases, because multiprecision floating-point arithmetic becomes necessary for high-precision Babai rounding. Concerning modulus waste, the encryption needs a P as large as $O(w^{-1+4/k})$ to be effective, which significantly reduces the usable ciphertext modulus in high-precision scenarios.

The high computational overhead from re-encoding the modulus-switching noise motivated us to find a lightweight alternative for plaintext encoding, which led us to discover Lemma 4 and to design the F-encoding algorithm. Fortunately, fusing this encoding format into ciphertext decryption not only enables noise re-encoding for free but also avoids any modulus waste.

References

1. Agrawal, R., Ahn, J.H., Bergamaschi, F., Cammarota, R., Cheon, J.H., D. M. de Souza, F., Gong, H., Kang, M., Kim, D., Kim, J., de Lassus, H., Park, J.H., Steiner, M., Wang, W.: High-precision rns-ckks on fixed but smaller word-size architectures: theory and application. In: Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. p. 23–34. WAHC '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3605759.3625257, https://doi.org/10.1145/3605759.3625257
2. Asharov, G., Jain, A., López-Alt, A., Tromer, E., Vaikuntanathan, V., Wichs, D.: Multiparty computation with low communication, computation and interaction via threshold FHE. In: Pointcheval, D., Johansson, T. (eds.) Advances in Cryptology – EUROCRYPT 2012. Lecture Notes in Computer Science, vol. 7237, pp. 483–501. Springer Berlin Heidelberg, Germany, Cambridge, UK (Apr 15–19, 2012). https://doi.org/10.1007/978-3-642-29011-4_29
3. Bae, Y., Cheon, J.H., Kim, J., Stehlé, D.: Bootstrapping bits with CKKS. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, Part II. Lecture Notes in Computer Science, vol. 14652, pp. 94–123. Springer, Cham, Switzerland, Zurich, Switzerland (May 26–30, 2024). https://doi.org/10.1007/978-3-031-58723-8_4
4. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with CKKS. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024, Part I. Lecture Notes in Computer Science, vol. 15484, pp. 330–360. Springer, Singapore, Singapore, Kolkata, India (Dec 9–13, 2024). https://doi.org/10.1007/978-981-96-0875-1_11
5. Blindenbach, J., Cheon, J.H., Gürsoy, G., Kang, J.: On the overflow and p-adic theory applied to homomorphic encryption. In: Dolev, S., Elhadad, M., Kutyłowski, M., Persiano, G. (eds.) Cyber Security, Cryptology, and Machine Learning. pp. 268–279. Springer Nature Switzerland, Cham (2025)
6. Boneh, D., Kim, J.: Homomorphic encryption for large integers from nested residue number systems. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025, Part III. Lecture Notes in Computer Science, vol. 16002, pp. 338–370. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2025). https://doi.org/10.1007/978-3-032-01881-6_11
7. Bonte, C., Bootland, C., Bos, J.W., Castryck, W., Iliashenko, I., Vercauteren, F.: Faster homomorphic function evaluation using non-integral base encoding. In:

Fischer, W., Homma, N. (eds.) Cryptographic Hardware and Embedded Systems – CHES 2017. Lecture Notes in Computer Science, vol. 10529, pp. 579–600. Springer, Cham, Switzerland, Taipei, Taiwan (Sep 25–28, 2017). https://doi.org/10.1007/978-3-319-66787-4_28

8. Bootland, C., Castryck, W., Iliashenko, I., Vercauteren, F.: Efficiently processing complex-valued data in homomorphic encryption. Journal of Mathematical Cryptology **14**(1), 55–65 (2020). https://doi.org/doi:10.1515/jmc-2015-0051, https://doi.org/10.1515/jmc-2015-0051
9. Bossuat, J.P., Cammarota, R., Chillotti, I., Curtis, B.R., Dai, W., Gong, H., Hales, E., Kim, D., Kumara, B., Lee, C., Lu, X., Maple, C., Pedrouzo-Ulloa, A., Player, R., Polyakov, Y., Lopez, L.A.R., Song, Y., Yhee, D.: Security guidelines for implementing homomorphic encryption. IACR Communications in Cryptology (CiC) **1**(4), 26 (2024). https://doi.org/10.62056/anxra69p1
10. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) Advances in Cryptology – CRYPTO 2012. Lecture Notes in Computer Science, vol. 7417, pp. 868–886. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 19–23, 2012). https://doi.org/10.1007/978-3-642-32009-5_50
11. Brakerski, Z., Friedman, O., Golan, D., Gurny, A., Mutzari, D., Sheinfeld, O.: REFHE: Fully homomorphic ALU. In: Daemen, J., Thomé, E. (eds.) Advances in Cryptology – EUROCRYPT 2026, Part IV. Lecture Notes in Computer Science, vol. 16545, pp. 183–213. Springer, Cham, Switzerland, Rome, Italy (May 10–14, 2026). https://doi.org/10.1007/978-3-032-25327-9_7
12. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Goldwasser, S. (ed.) ITCS 2012: 3rd Innovations in Theoretical Computer Science. pp. 309–325. Association for Computing Machinery, Cambridge, MA, USA (Jan 8–10, 2012). https://doi.org/10.1145/2090236.2090262
13. Castryck, W., Iliashenko, I., Vercauteren, F.: Homomorphic SIM^{2}D operations: Single instruction much more data. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018, Part I. Lecture Notes in Computer Science, vol. 10820, pp. 338–359. Springer, Cham, Switzerland, Tel Aviv, Israel (Apr 29 – May 3, 2018). https://doi.org/10.1007/978-3-319-78381-9_13
14. Cha, G., Park, D., Lee, J.W.: Improved radix-based approximate homomorphic encryption for large integers via lightweight bootstrapped digit carry. In: Daemen, J., Thomé, E. (eds.) Advances in Cryptology – EUROCRYPT 2026, Part IV. Lecture Notes in Computer Science, vol. 16545, pp. 243–273. Springer, Cham, Switzerland, Rome, Italy (May 10–14, 2026). https://doi.org/10.1007/978-3-032-25327-9_9
15. Cha, H., Hwang, I., Min, S., Seo, J., Song, Y.: MatriGear: Accelerating authenticated matrix triple generation with scalable prime fields via optimized HE packing. In: Blanton, M., Enck, W., Nita-Rotaru, C. (eds.) 2025 IEEE Symposium on Security and Privacy. pp. 2453–2471. IEEE Computer Society Press, San Francisco, CA, USA (May 12–15, 2025). https://doi.org/10.1109/SP61157.2025.00174
16. Checri, M., Sirdey, R., Boudguiga, A., Bultel, J.P.: On the practical CPA^D security of "exact" and threshold FHE schemes and libraries. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part III. Lecture Notes in Computer Science, vol. 14922, pp. 3–33. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68382-4_1

17. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019: 26th Conference on Computer and Communications Security. pp. 395–412. ACM Press, London, UK (Nov 11–15, 2019). https://doi.org/10.1145/3319535.3363207
18. Chen, H., Iliashenko, I., Laine, K.: When heaan meets fv: A new somewhat homomorphic encryption with reduced memory overhead. In: Paterson, M.B. (ed.) Cryptography and Coding. pp. 265–285. Springer International Publishing, Cham (2021)
19. Chen, H., Laine, K., Player, R., Xia, Y.: High-precision arithmetic in homomorphic encryption. In: Smart, N.P. (ed.) Topics in Cryptology – CT-RSA 2018. Lecture Notes in Computer Science, vol. 10808, pp. 116–136. Springer, Cham, Switzerland, San Francisco, CA, USA (Apr 16–20, 2018). https://doi.org/10.1007/978-3-319-76953-0_7
20. Chen, Y.: Réduction de réseau et sécurité concrète du chiffrement complètement homomorphe. Ph.D. thesis, Université Paris Diderot (2013), http://www.theses.fr/2013PA077242, thèse de doctorat dirigée par Nguyen, Phong Q. Informatique Paris 7 2013; 2013PA077242
21. Cheon, J.H., Cho, W., Kim, J., Stehlé, D.: Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023: 30th Conference on Computer and Communications Security. pp. 696–710. ACM Press, Copenhagen, Denmark (Nov 26–30, 2023). https://doi.org/10.1145/3576915.3623086
22. Cheon, J.H., Choe, H., Kang, M., Kim, J., Kim, S., Mono, J., Noh, T.: Grafting: Decoupled scale factors and modulus in RNS-CKKS. In: Huang, C.Y., Chen, J.C., Shieh, S.P., Lie, D., Cortier, V. (eds.) ACM CCS 2025: 32nd Conference on Computer and Communications Security. pp. 1098–1112. ACM Press, Taipei, Taiwan (Oct 13–17, 2025). https://doi.org/10.1145/3719027.3765083
23. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) Advances in Cryptology – ASIACRYPT 2017, Part I. Lecture Notes in Computer Science, vol. 10624, pp. 409–437. Springer, Cham, Switzerland, Hong Kong, China (Dec 3–7, 2017). https://doi.org/10.1007/978-3-319-70694-8_15
24. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: Cheon, J.H., Takagi, T. (eds.) Advances in Cryptology – ASIACRYPT 2016, Part I. Lecture Notes in Computer Science, vol. 10031, pp. 3–33. Springer Berlin Heidelberg, Germany, Hanoi, Vietnam (Dec 4–8, 2016). https://doi.org/10.1007/978-3-662-53887-6_1
25. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. Journal of Cryptology **33**(1), 34–91 (Jan 2020). https://doi.org/10.1007/s00145-019-09319-x
26. Choe, H., Kim, J., Stehlé, D., Suvanto, E.: Leveraging discrete CKKS to bootstrap in high precision. In: Huang, C.Y., Chen, J.C., Shieh, S.P., Lie, D., Cortier, V. (eds.) ACM CCS 2025: 32nd Conference on Computer and Communications Security. pp. 1083–1097. ACM Press, Taipei, Taiwan (Oct 13–17, 2025). https://doi.org/10.1145/3719027.3765080
27. Costache, A., Smart, N.P., Vivek, S.: Faster homomorphic evaluation of discrete fourier transforms. In: Kiayias, A. (ed.) Financial Cryptography and Data Security. pp. 517–529. Springer International Publishing, Cham (2017)

28. Cramer, R., Ducas, L., Peikert, C., Regev, O.: Recovering short generators of principal ideals in cyclotomic rings. In: Fischlin, M., Coron, J.S. (eds.) Advances in Cryptology – EUROCRYPT 2016, Part II. Lecture Notes in Computer Science, vol. 9666, pp. 559–585. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). https://doi.org/10.1007/978-3-662-49896-5_20
29. Drucker, N., Moshkowich, G., Pelleg, T., Shaul, H.: BLEACH: Cleaning errors in discrete computations over CKKS. Journal of Cryptology **37**(1), 3 (Jan 2024). https://doi.org/10.1007/s00145-023-09483-1
30. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology – EUROCRYPT 2015, Part I. Lecture Notes in Computer Science, vol. 9056, pp. 617–640. Springer Berlin Heidelberg, Germany, Sofia, Bulgaria (Apr 26–30, 2015). https://doi.org/10.1007/978-3-662-46800-5_24
31. Dumezy, J.: Fully homomorphic encryption for small batches of complex numbers. Cryptology ePrint Archive, Paper 2026/2513 (2026), https://eprint.iacr.org/2026/2513
32. Dumezy, J.: Sparse key estimate. https://github.com/jdumezy/sparse-key-estimate (2026)
33. Eriksson, J., Koivunen, V.: Complex random vectors and ica models: identifiability, uniqueness, and separability. IEEE Transactions on Information Theory **52**(3), 1017–1029 (2006). https://doi.org/10.1109/TIT.2005.864440
34. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144 (2012), https://eprint.iacr.org/2012/144
35. Fan, K.: On a theorem of Weyl concerning eigenvalues of linear transformations I. Proceedings of the National Academy of Sciences of the United States of America **35**(11), 652–655 (1949). https://doi.org/10.1073/pnas.35.11.652
36. Gao, M., Zheng, H.: Fhe for simd arithmetic logic units with amortized o(1) bootstrapping per ciphertext. In: Heninger, N., Rosulek, M. (eds.) Advances in Cryptology – CRYPTO 2026. pp. 299–330. Springer Nature Switzerland, Cham (2026)
37. Geelen, R., Vercauteren, F.: Fully homomorphic encryption for cyclotomic prime moduli. In: Fehr, S., Fouque, P.A. (eds.) Advances in Cryptology – EUROCRYPT 2025, Part III. Lecture Notes in Computer Science, vol. 15603, pp. 366–397. Springer, Cham, Switzerland, Madrid, Spain (May 4–8, 2025). https://doi.org/10.1007/978-3-031-91131-6_13
38. Geelen, R., Vercauteren, F.: Better GBFV bootstrapping and faster encrypted edit distance computation. IACR Communications in Cryptology (CiC) **2**(4), 35 (2026). https://doi.org/10.62056/a3n56chdj
39. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Mitzenmacher, M. (ed.) 41st Annual ACM Symposium on Theory of Computing. pp. 169–178. ACM Press, Bethesda, MD, USA (May 31 – Jun 2, 2009). https://doi.org/10.1145/1536414.1536440
40. Gentry, C., Lee, Y.: Fully homomorphic encryption for matrix arithmetic. In: Heninger, N., Rosulek, M. (eds.) Advances in Cryptology – CRYPTO 2026. pp. 363–395. Springer Nature Switzerland, Cham (2026)
41. Han, K., Ki, D.: Better bootstrapping for approximate homomorphic encryption. In: Jarecki, S. (ed.) Topics in Cryptology – CT-RSA 2020. Lecture Notes in Computer Science, vol. 12006, pp. 364–390. Springer, Cham, Switzerland, San Francisco, CA, USA (Feb 24–28, 2020). https://doi.org/10.1007/978-3-030-40186-3_16

42. Hou, J., Jiang, H., Ogilvie, T.: Careful with the ring! concrete hardness gaps between lwe and mlwe. In: Heninger, N., Rosulek, M. (eds.) Advances in Cryptology – CRYPTO 2026. pp. 462–497. Springer Nature Switzerland, Cham (2026)
43. Hu, Z., Liang, H., Wang, X., Xie, X., Yang, K., Yu, Y., Zhang, W.: Ajax: Fast threshold fully homomorphic encryption without noise flooding. In: 35th USENIX Security Symposium (USENIX Security 26). USENIX Association, Baltimore, MD (Aug 2026), https://www.usenix.org/conference/usenixsecurity26/presentation/hu-zhenkai
44. Kannan, R.: Minkowski's convex body theorem and integer programming. Mathematics of Operations Research **12**(3), 415–440 (1987). https://doi.org/10.1287/moor.12.3.415, https://doi.org/10.1287/moor.12.3.415
45. Kim, J.: Efficient homomorphic integer computer from CKKS. IACR Transactions on Cryptographic Hardware and Embedded Systems **2025**(4), 873–898 (2025). https://doi.org/10.46586/tches.v2025.i4.873-898
46. Kim, J.: Faster homomorphic integer computer. IACR Transactions on Cryptographic Hardware and Embedded Systems **2026**(3), 1128–1151 (Jul 2026). https://doi.org/10.46586/tches.v2026.i3.1128-1151, https://tches.iacr.org/index.php/TCHES/article/view/13141
47. Kluczniak, K., Santato, G.: On circuit private, multikey and threshold approximate homomorphic encryption. IACR Communications in Cryptology (CiC) **2**(1), 9 (2025). https://doi.org/10.62056/a69qgyl7s
48. Lattigo v6. Online: https://github.com/tuneinsight/lattigo (Aug 2024), ePFL-LDS, Tune Insight SA
49. Li, B., Micciancio, D.: On the security of homomorphic encryption on approximate numbers. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021, Part I. Lecture Notes in Computer Science, vol. 12696, pp. 648–677. Springer, Cham, Switzerland, Zagreb, Croatia (Oct 17–21, 2021). https://doi.org/10.1007/978-3-030-77870-5_23
50. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) Advances in Cryptology – EUROCRYPT 2010. Lecture Notes in Computer Science, vol. 6110, pp. 1–23. Springer Berlin Heidelberg, Germany, French Riviera (May 30 – Jun 3, 2010). https://doi.org/10.1007/978-3-642-13190-5_1
51. Marcus, D.A.: Number Fields. Universitext, Springer, New York, NY, 1 edn. (1977). https://doi.org/10.1007/978-1-4684-9356-6
52. Ogilvie, T.: IND-CPA-D and KR-D security with reduced noise from the HintLWE problem. In: Hanaoka, G., Yang, B.Y. (eds.) Advances in Cryptology – ASIACRYPT 2025, Part VII. Lecture Notes in Computer Science, vol. 16251, pp. 263–295. Springer, Singapore, Singapore, Melbourne, VIC, Australia (Dec 8–12, 2025). https://doi.org/10.1007/978-981-95-5122-4_9
53. Ollila, E.: On the circularity of a complex random variable. IEEE Signal Processing Letters **15**, 841–844 (2008). https://doi.org/10.1109/LSP.2008.2005050
54. Schnorr, C.P., Euchner, M.: Lattice basis reduction: improved practical algorithms and solving subset sum problems. Math. Program. **66**(2), 181–199 (Sep 1994). https://doi.org/10.1007/BF01581144, https://doi.org/10.1007/BF01581144
55. Schnorr, C.P.: Lattice reduction by random sampling and birthday methods. In: Alt, H., Habib, M. (eds.) STACS 2003. pp. 145–156. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
56. development team, T.F.: fpylll, a Python wrapper for the fplll lattice reduction library, Version: 0.6.4 (2025), https://github.com/fplll/fpylll, available at https://github.com/fplll/fpylll

57. Weber, H.: Theorie der Abel'schen Zahlkörper. Acta Mathematica **8**(none), 193 – 263 (1886). https://doi.org/10.1007/BF02417089, https://doi.org/10.1007/BF02417089